REN-209 Putting it all together.

Complex Configurations

Understanding the Problem

Rhapsody provides a robust and scalable environment, able to handle large and complex configurations just as well as small configurations.

While there are numerous small sites utilizing Rhapsody in a niche configuration (fewer than 10 communication points), most sites rapidly build to mid-sized configurations (100 to 300 communication points), and large configurations (in excess of 3,000 communication points and 1,000 routes) are not uncommon.

The ease of implementing interfaces frequently results in incremental growth in the complexity of a configuration, and may result in environments which are difficult to maintain if appropriate structures and standards are not enforced from the start.

Setting the Bar - Best Practices

While project pressures often make it tempting to simply start the process of building interfaces, it is better to first build an overview of the likely end point and long term state for the configuration. This will enable the development of standards to guide the configuration structure. Note that Best Practice and site standards will always be considered guidelines as the configuration develops.

Attention should be given to:

- Interface maintainability.
- Organizational structures and constraints; in particular, the requirement that portions of the configuration be managed by separate dedicated teams.
- Types of communication points being utilized.
- Constraints on system connections (both inbound and outbound).
- Messaging standards which will need to be supported.
- Role of Rhapsody for each interface; is it a conduit, or does Rhapsody take responsibility for messages on the interface?
- Monitoring hierarchies.
- Error management.
- User responsibilities.

This list provides the basis for establishing Best Practice guidelines for the site, including:

- Structure of the configuration (folder hierarchies and responsibilities).
- Performance constraints.
- Naming conventions for routes, communication points and filters.
- Port assignments (TCP and related connections).
- Management of messaging standards (for example, should each route be treated as independent or are there advantages to be gained by implementing a core message standard to which all messages are translated on input and from which all messages are translated on output).
- Watchlist structures and scheduling.
- Interface guidelines, including the use of complex or cascaded routes.
- Management policies, including user management and responsibilities and password management.
- Documentation standards.

Many of these topics have been encountered in earlier modules of this course, but the balance of this module will address key issues.

Maintainability

The two primary goals for building a configuration are that the interfaces perform correctly according to the specifications and that they are maintainable.

Maintainability is a rather nebulous concept, but fundamentally means that an analyst can understand the purpose and the operation of the interface quickly and without spending large amounts of time wading through documentation, which is often unstructured and poorly organized.

Each interface should therefore be well laid out and effectively self-documenting, particularly where there are unusual or hidden elements (for example dependency on message fields used for a non-standard purpose or where a feedback loop in the configuration is necessary).

To achieve the maintainability goal, interfaces should therefore:

- Be effectively self-documenting; component naming conventions should be used which indicate the role / purpose of the component in the interface.
- Be clearly laid out in a visual sense; the working framework of the route canvas provides a flow from Input to the left of the canvas to Output to the right. Each interface should essentially flow from left to right, avoiding loops back to the left

and avoiding cross-over of links where possible (there will always be unavoidable exceptions to these guidelines).

- Use the route Notes for documentation. It is best to use a template to record author / date / updates, to describe the purpose and any particular features of the interface which may be unclear from the canvas view. This is particularly important where a collector may be used to aggregate results of multiple paths in the route.

The final arbiter of maintainability is to determine if there is sufficient information for you to understand the interface if you return to it 12 months after last reviewing or working on it.

Testing... Testing... Testing...

The final stage of implementation is to ensure the configuration is working correctly according to the specification.

Achieving this goal successfully requires attention at all stages of a project to ensure that the goals are well understood. It is preferable that issues are trapped during the testing rather than being revealed once the configuration has been placed into production.

Attention should be paid in particular to the following:

- Specifications, particularly messaging standards; this requires the specifications and standards documents be available and well known, and that any deviations, particularly from standards, are known and documented.
- Data envelope; testing requires that the full envelope of messaging data be known or at least anticipated (a common fault with production systems is that the data envelope is broader than anticipated and the configuration is unable to cope).
- Management of errors. An error management strategy should be established early in the project so that errors are proactively handled, and the Error Queue is used to handle only true exceptions. Expected exceptions should be handled on the routes with effective notifications and handling of the messages.
- Incremental testing. The filter testing framework should be utilized to ensure that the filter operation is correct before finalizing the configuration - the final testing should then effectively only require end-to-end testing and validation. Filter testing is discussed later in this module.

Building a Test Framework

During the development phase of a project, it is rare to have access to the message feed for an interface (and in many cases, the volume of messaging from that feed would be too large to manage effectively).

It is therefore useful to utilize simple and more controlled methods for supplying messages to the interface. These might include:

- Using a Directory communication point.
- Using a Timer communication point; this has the disadvantage that the message is always the same unless you provide additional functionality on the route.
- Using a TCP framework (requires building an additional route or additional pathways in the route).
- Using the EDI Explorer to provide an external TCP feed; this may introduce complexities into the TCP transfer and become cumbersome if you wish to use more than a few distinct messages.

The methodology used for any particular project will depend on analyst preference, although you should ensure that the methodology chosen does not introduce artifacts into the testing.

During the final stages of testing, the range of messages handled should be extended by utilizing either a feed from the production environment or a large extract of messaging from the production environment.

Connecting Routes

Connection Methods

Generally, a complex solution will be divided into a number of routes along functional lines. A route may also need to be split into multiple routes when the number of components on the route becomes so large that the visual layout becomes complex, or that the complexity of connections between components becomes sufficiently high that any single message path is difficult to determine. For example, an ADT message coming from an EMR system could be needed by a Lab system, a Radiology system and a Pharmacy system. Each system may have specific transformations that must be made to the same message in order for the message to be accepted.

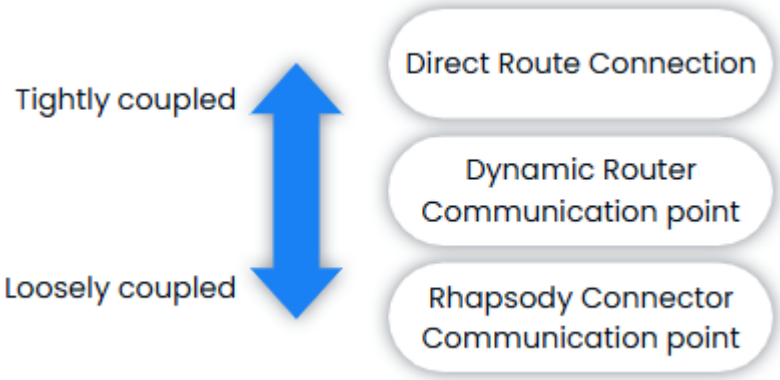
Although it may be tempting to handle the visual complexity of a complex route by expanding the route canvas in the IDE to full screen, it is better to split the interface into logical blocks across multiple routes.

Interconnect Methods

The two most common methods used by Rhapsody to provide multiple downstream routes for the same messages are the Direct Route Connection and the Dynamic Router

communication points. The third method, the Rhapsody Connector communication point, is only used when sending messages between different Rhapsody engines.

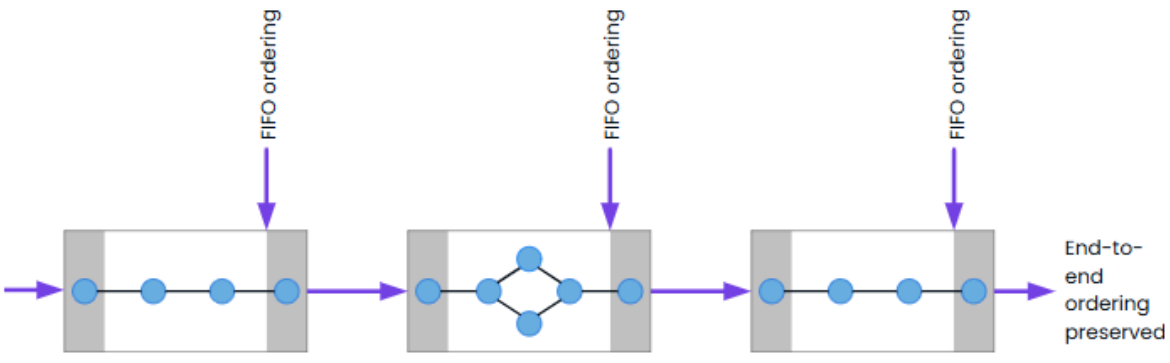
They are listed below and ordered by how tightly coupled they are. Two routes might be tightly coupled when a single configuration has been split into two routes for clarity. In comparison, two routes might be loosely coupled, where one route received messages and the second sends messages to an independent downstream system.



It is much easier to migrate the configuration for loosely coupled routes. The migration is more challenging the more interdependent the routes are.

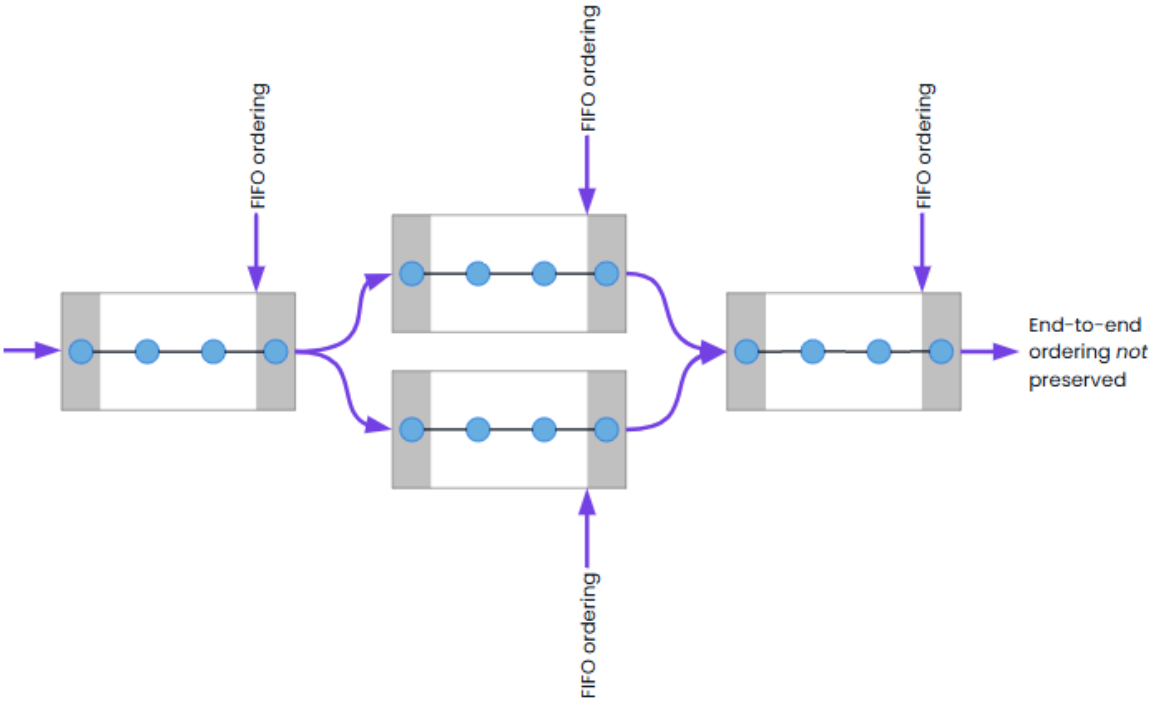
FIFO

Messages are reordered according to a route's FIFO settings at the exit of each route. This ensures that messages leave a route in the same order that they arrived in. When routes are interconnected, this reordering check will occur multiple times during the processing of a message. If all messages follow the same path between routes, message ordering can be preserved end-to-end.



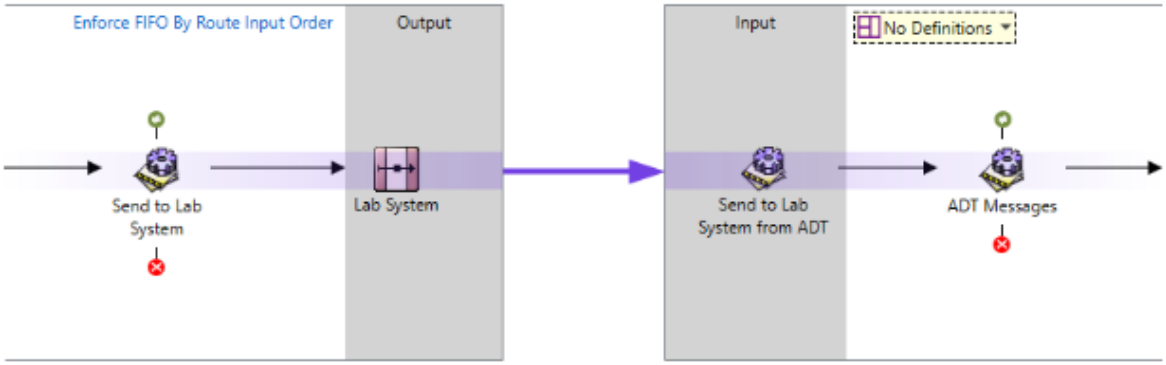
Three connected routes with FIFO ordering occurring at the output of each route. FIFO is preserved end-to-end

If some messages follow a different sequence of routes though the solution, those messages may arrive out-of-order at the destination (all messages that follow the same sequence of routes will arrive in order).



Connected routes with multiple paths. FIFO is not preserved between messages that traverse different routes

Direct Route Connection



Two route connected via a Direct Route Connection

Routes within the same locker may be interconnected by dragging the destination route into the Output section of the workspace canvas for the source route. Connections are then established in the normal manner, although it is important to protect the inter-route connections. This is achieved by placing No-operation filters immediately before and after the interconnect; that is

- On the upstream route, place a No-operation filter as the final filter in the path; the connection between the filter and the route in the Output frame should be a simple connection (that is, without a condition).
- On the downstream route, place a No-operation filter as the first filter in the route.

This technique is referred to as bookending a route. This preserves and protects the connections from other changes to the routes.

There is no limit on the number of interconnected routes that can be placed on the Output section of the workspace. Multiple routes can be added to the Output section of the workspace by dragging the next destination route to the Output section of the workspace.

The advantage of using a direct route connection is that is quick and easy to implement. Reviewing is also easy, as double-clicking on the components on the input or output of a route will open the corresponding route.

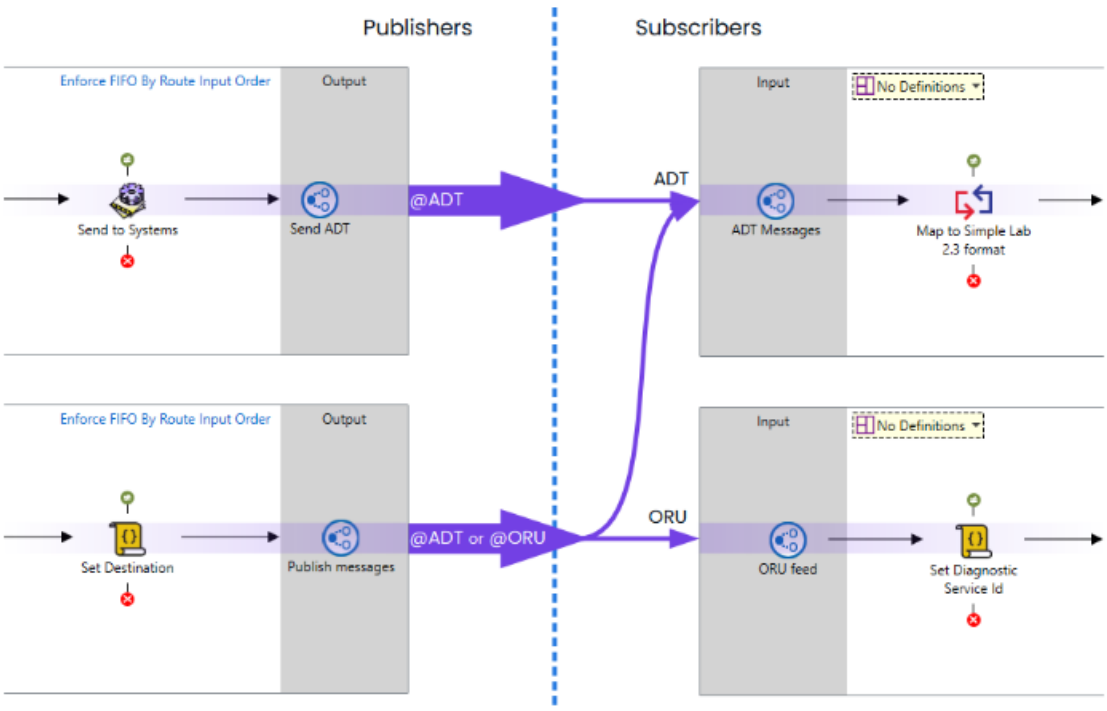
The disadvantage is that a direct connection may be fragile if not correctly bookended with No-operation filters. Consequently, the state of the connection should be checked whenever making a change to either route, and the connections re-confirmed on both routes after checking in. When exporting one route, any directly connected routes (and associated communication points) will also need to be exported. This can make migration of configuration more difficult.

This method works well when there are relatively few destination routes. Best practice is to use a Dynamic Router communication point as a publisher to the destination routes when there are enough interconnected routes to require scrolling to view all destination routes.

Dynamic Router Communication Point

The Dynamic Router Communication Point allows an output from one route to be connected to one or more route inputs somewhere else within the same Rhapsody configuration. Messages are sent to an instance of the dynamic router communication point running output mode. They immediately appear on any dynamic router communication points configured in input mode with matching configuration. The destination for the message is configured into the outbound communication point at configuration time or dynamically using a message property.

Using the dynamic router for routing messages can be described as the Publish-Subscribe method of routing messages. A dynamic router set to output mode will publish (send) the messages, while one or more destination dynamic routers set to input put mode can subscribe (receive) the messages.



A group of routes connected together via Dynamic Routers

The advantage of using a dynamic router is that a message can be sent to any dynamic router running in input mode anywhere in the engine. Like the direct connection, the message structure, properties, and path through the engine are preserved and can easily be traced using the Management Console. This more loosely coupled nature of the Dynamic Router means that the link between two routes is not broken when a component change occurs. This also means that exporting a route using Dynamic Router interconnects does not require any other routes to be exported. This makes a solution more modular and easier to upgrade. Another advantage of the Dynamic Router is that it supports a "Publish / Subscribe" mode of operation; that is, the upstream route publishes the message through the communication point, and subscription is achieved by downstream routes listing for messages tagged with the appropriate routing tag.

The disadvantage is of a dynamic router is that without a clear naming convention, it may not be obvious what the destination for a message sent to a dynamic router will be.

Configuring the Dynamic Router

For messages to be processed by a Dynamic Router communication point, at minimum there must be one Dynamic Router set to Output mode (publisher) and one or more Dynamic Routers set to Input mode (subscriber).

The target destination can be either configured statically in the configuration properties, or can be set dynamically using the router:Destination message property. Careful consideration needs to be given when choosing a Target name (destination) for Dynamic

Routers. You should choose a target name that will uniquely identify the messages for processing. A consistent naming convention is important.

Configuring the Publishing Dynamic Router

A Dynamic Router Communication Point when configured in Output mode will publish messages to all subscribing dynamic routers that are listening for messages with the appropriate destination.

The destination can be configured using either a target name or the name of the receiving component. A destination within the same locker can be specified using either the target name preceded with the @ symbol (for example, @myTarget), or the relative component path (for example, myFolder/Dynamic Router 1). A destination in another locker can be configured using either the target name preceded with the @ symbol, or the full component path preceded by the # symbol (for example, #myLocker/myFolder/Dynamic Router 1).

When using the relative component path to specify the receiving Dynamic Router, care should be taken renaming the communication point as this could break your configuration.

A destination must be considered valid in order for it to be used. A valid destination is:

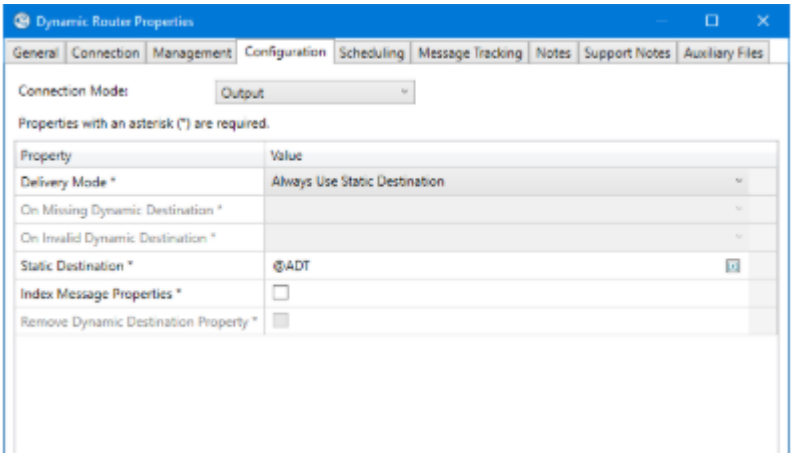
- An instance of the Dynamic Router communication point.
- Configured to run in input only mode.
- Attached to one or more routes.

If multiple destinations are identified for a message due to either multiple paths being provided or a target name being used by multiple Dynamic Router input communication points, all the evaluated destinations are validated. An error with any of these destinations triggers the configured error action and the message will not be sent to any of those destinations. This means that duplicate messages are avoided if the user subsequently fixes the message and re-sends it.

Using a Static Destination

When using the Dynamic Router statically, the **Delivery Mode** property is set to **Always Use Static Destination**.

The **Static Destination** property determines the destination for all messages irrespective of any value set in the router:Destination message property.



Using a Dynamic Destination

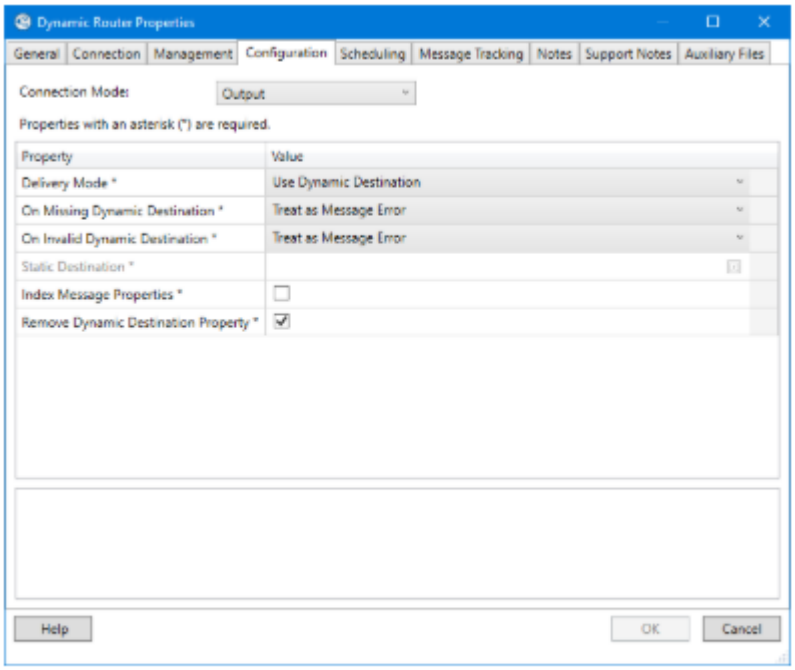
When using the Dynamic Destination for the Delivery Mode, the communication point requires a message property called router:Destination to contain the value of the target destination. The router:Destination property must be set before the message reaches the publishing Dynamic Router. This value could be different for each message, depending on business rules or processing logic.

The Dynamic Router can deliver to multiple destinations in two ways: firstly, by using a **Target Name** (for example, @myTarget) which multiple Dynamic Router input communication points have configured as a non-unique target name. Alternatively, multiple destinations can be selected dynamically using the router:Destination message property with a property list containing multiple target names or component paths.

Property lists are message properties containing multiple values and are set using the addPropertyValue() function in JavaScript.

When configured to use a dynamic destination, the following properties need to be configured:

- **On Missing Dynamic Destination** - Determines the behavior when a message is sent to the Dynamic Router missing the router:Destination property or the property has no value. When configuring this setting, you should consider what needs to happen with these messages. The two available settings for this property are:
    - Treat as Message Error
    - Treat as Connection Error
    - Use Static Destination



Dynamic Router configured to output messages to a dynamic destination

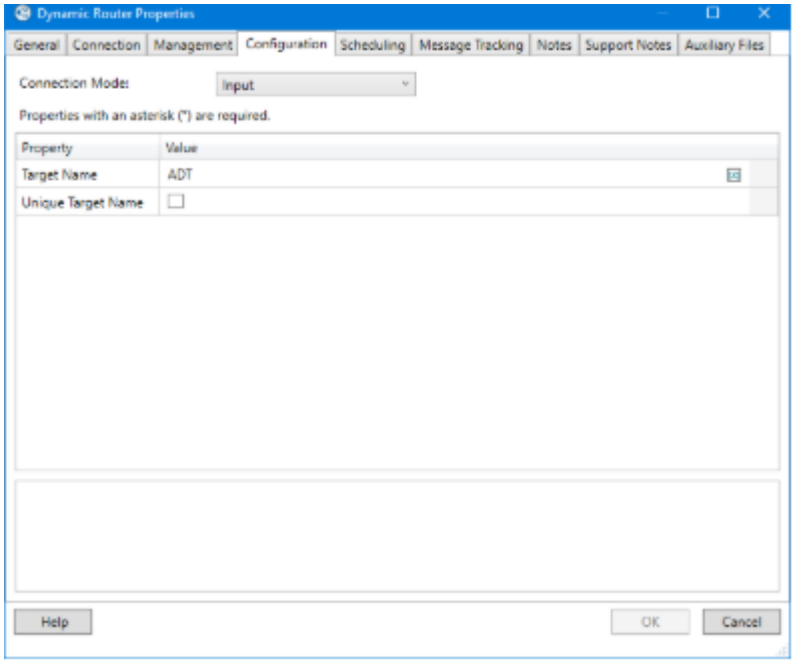
- **On Invalid Dynamic Destination** - Determines the behavior when a message is sent to the Dynamic Router containing a router:Destination property that does not match any subscribing Dynamic Routers. Configuration for this setting will likely mirror the setting for the On Missing Dynamic Destination property.
- **Remove Dynamic Destination Property** - This property allows you to remove the target destination as the message is being sent to the subscriber. This can help eliminate the possibility of inadvertently creating an infinite loop. Using this option can slightly decrease the Dynamic Router performance. Removing the target destination property can also be done by using a filter, such as a JavaScript filter or a Property Population filter.
- **Index Message Properties** - This property indicates whether message properties are indexed after sending this message out from the Dynamic Router communication point. Normally this is always done when messages leave Rhapsody via a communication point. However, as the Dynamic Router is a mid-point rather than an exit, it is possible that indexing of message properties is not required here as it will just increase disk space usage and performance for a searching scenario that is unlikely to take place. Note that disabling message property indexing here does not affect whether or not message properties are indexed when they finally leave Rhapsody - it just affects whether they are indexed at this communication point or not.

- It is possible to inadvertently create an infinite loop using Dynamic Routers. When a subscribing route receives a message that was sent using the `router:Destination` property and the subscribing route also has a Dynamic Router as an output, it is important that the `router:Destination` property be updated to a new destination or removed. This can be done by setting the **Remove Dynamic Destination** Property to **True** on the original publishing Dynamic Router or by deleting or updating the property on each subscribing route.
- Any input message that is processed though a Dynamic Router more than 50 times within an hour is considered to be a stuck in an infinite loop and will be sent to the Error Queue.

- **Configuring the Subscribing Dynamic Router**
- A Dynamic Router Communication Point configured in Input mode needs to be created and added to all subscribing routes that will be receiving messages from a publishing route.

- The **Target Name property** specifies the target name that will be subscribed to.

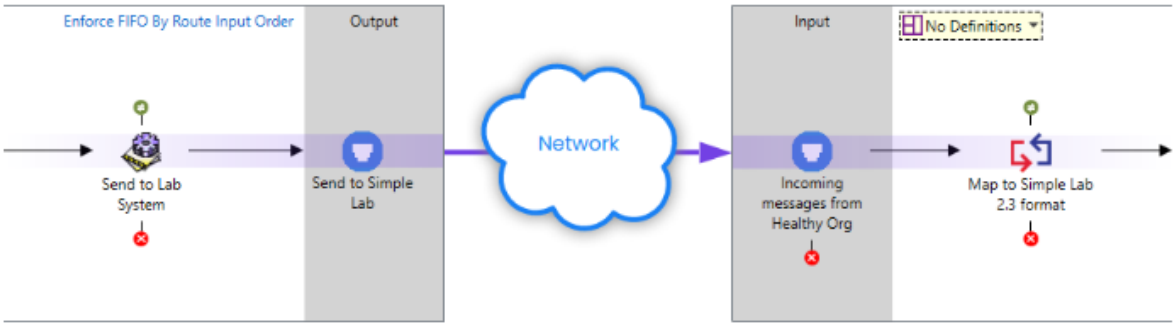
- The **Unique Target Name** property should be set to true when we only want one Dynamic Router in the configuration to be able to receive messages with a particular tag. When used, attempting to configure a second communication point using the same target name will fail.



Dynamic Router configured in input mode

Connecting to other Rhapsody Engines

The Rhapsody Connector is designed to send messages between two different Rhapsody engines. When used, the message body along with any message properties will be sent to the destination engine. The Rhapsody Connector is useful when a solution expands to include multiple Rhapsody servers to cover multiple sites (each with their own server).



Rhapsody Connector communication points being used to send messages between two Engines

The Rhapsody Connector communication point streams messages and all their message properties via a TCP connection. In addition, if the Rhapsody engine is aware of the character encoding of the message (i.e., it has been explicitly set by a character translation filter or JavaScript filter) then it is transmitted along with the message body.

The Rhapsody Connector protocol supports some additional functionality to improve the reliability of these communications. For example:

- A protocol acknowledgement mechanism is automatically included to ensure that messages have been completely received by the remote system before being regarded as sent.
- Keep alive requests and responses that are used at a configurable interval to ensure the connection is still valid, and has not been closed by an overly zealous firewall.
- Connection capabilities to prevent systems from accidentally connecting to each other.

The advantage of the Rhapsody Connector is that it allows different Rhapsody engines to exchange messages in a secure, reliable fashion.

The disadvantages associated with the Rhapsody Connector are seen when it is used to connect routes on the same Rhapsody host. In such a case, the Rhapsody Connector communication point will be less efficient than either a Direct Route Connection or Dynamic Router. This is due to the overhead involved in sending a message to a communication point that then loops through the network stack on the host operating system and back to a second communication point in Rhapsody. Additionally, it is not possible to track messages using the **Management Console** through this communication point, making tracing more difficult.

Connections using other Communication Points

While other communication points can be used to connect routes in Rhapsody, they don't share the advantages of the various options discussed above. Occasionally considered communication points include:

- **TCP** communication points
- **Directory** communication point

Communication points such as the **TCP** and **Directory** communication points present a number of disadvantages when compared to the previously discussed interconnect methods. Only the message body is passed across the connection; all properties and processing history are lost, this makes it hard to retain information stored in message properties. Nor is there a guarantee of delivery, thus message tracking must also be configured to make the solution robust.

These communication points should only be used when sending messages to an instance of Rhapsody operated by an independent organization.

Message Collectors

Overview

Message Collectors provide the ability to manage sequences, groups, and batches of messages input to a filter. The collector operates at the input to the filter and supplies the resultant set of messages to the filter on which the collector is configured.

All filters may have collector criteria set, but only a few filters support operations on the resultant message set (most commonly the Batch / De-batch, Zip / Unzip, & JavaScript filters). If a filter does not group the message set into a single message, they are handled by the filter and output in the order in which they were received from the set. The balance of the filters operate on the members of the message set as individual messages rather than as the collection, and output them in the sequence received.

Message Sequencing

Message Sequencing ensures that message ordering constraints are met and supported by the following collectors:

- **Input Collation**: orders messages based on the input path to the filter, and releases the messages in the specified order when a message has been received from all inputs.
- **Message Set**: orders messages based on a message property, and releases the set when the correct number of messages for the set are received.
- **Message Set Trigger**: orders messages based on a message property, and releases the set when a trigger message is received and all members of the set are available.

Message Grouping

Message Grouping collects messages and releases them in the order they are received when the release criteria are satisfied;

- **Property**: groups messages based on a message property and releases the group when the timeout period expires.
- **Size and/or Time**: holds messages until the size or time criterion is exceeded.
- **Trigger**: hold messages until a message is received with the Trigger property set to any value, and then releases the group.

Message Delaying

Message Delaying collectors delay individual messages without making any attempt to group or sequence them;

- **Timed Hold**: holds messages until the time criterion is exceeded.

Common Usage Scenarios

- Grouping a set of messages into a single zip file using the Zip / Unzip filter before being sent to a destination system.
- Grouping an HL7 message with the response from a web service lookup, using a JavaScript filter to merge the results from the lookup into the HL7 message.
- Delaying the processing of individual messages for 10 seconds to ensure they reach the destination system after update by some other process.

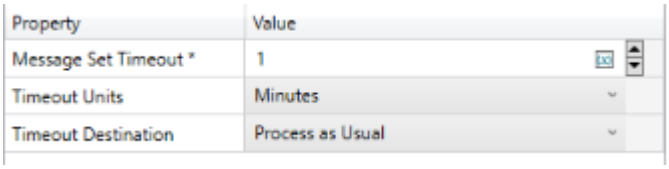
Timeouts in Message Collectors

Message Collectors provide the ability to group messages by means of a range of criteria, and consequently offer the potential for a message to be delayed by a collector indefinitely. This may have less than desirable effects on message processing, for example, where FIFO constraints result in the backing up of messages due to the delayed message.

Each of the collector models therefore includes the ability to set a time based constraint which will override other constraints if they are not met within the time period.

The message timeout may be configured for

- The timeout period
- The units for the timeout period (seconds, minutes, hours, or days)
- The action to take if the timeout occurs (Normal processing, process via no-match connector, process via error connector).



If **Process as Normal** is selected, a partial set of messages may be presented to the filter associated with the collector. For some filters (in particular, the JavaScript filter), this frequently results in errors. In these cases, use of the no-match or error connectors as a timeout destination can result in a more robust solution.

The timeout behavior may be overridden by setting the timeout period to zero (0), but this is not recommended.

We recommend that an appropriate Message Set Timeout value is configured for all message collectors.

Message Collectors and route FIFO rules

On a route that has FIFO enabled, all messages will exit the route in the same order they arrived in, even when a collector delays a message. This is particularly important when a collector delays a subset of messages on a route, which can happen when some messages needed to make a set for the collector arrive slowly, or when the timed hold collector delays some messages. Messages that bypass the collector will be delayed on exit from the route until the delayed messages exit the route.

Where it is essential that a message collector is allowed to alter message order, the collector must be placed on a route with FIFO rules disabled. When other parts of the solution require the use of FIFO, changed routes or dynamic routes should be used to link the various FIFO and non-FIFO sections of the configuration.

Message Sequencing

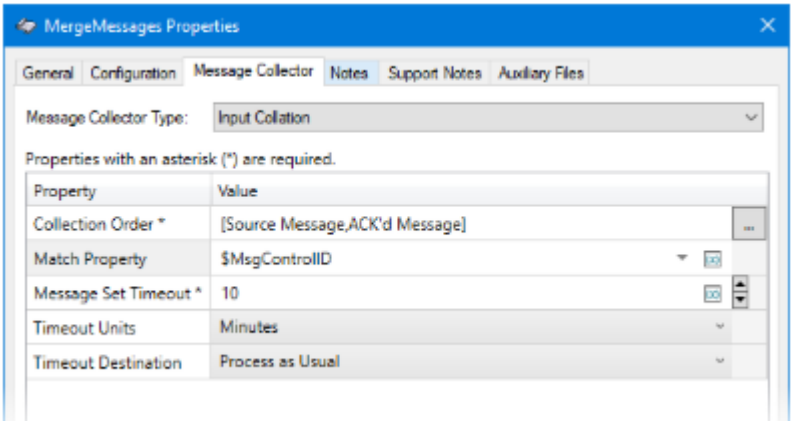
The Message Sequencing collectors permit the messages arriving at the filter to be ordered for further processing to ensure correct message sequencing. Messages may be ordered by the path on which they are received by the filter or by a message property.

Input Collation

The **Input Collation** collector orders the messages by the path on which they arrive at the filter. This may be necessary, for example, where multiple operations are required on a message which are carried out in parallel, or in situations where messages are received on multiple inputs but require sequencing for downstream processing.

The Input Collation collector type requires the following properties to be set:

- **Collection Order**: defines the order in the set as well as the message set itself (that is, the set comprises one message from each path included, ordered by the path order).
- **Match Property**: provides additional structure to the set by providing a property whose value must be matched for a message to be included in the set (otherwise, messages are simply grouped by arrival sequence).

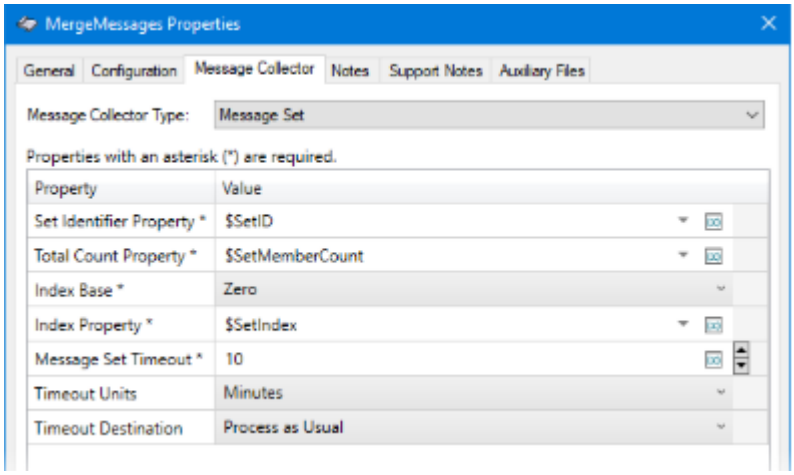


Message Set

The Message Set collector utilizes message properties to define the set and order in the set; these will typically be set either by message content or upstream filter processing.

The Message Set collector type requires the following properties to be set:

- **Set Identifier Property**: identifies the set of which the message is a member.
- **Total Count Property**: identifies the total number of messages contained in the set.
- **Index Base**: provides the base value for the Index Property.
- **Index Property**: defines the position in the set for the message.

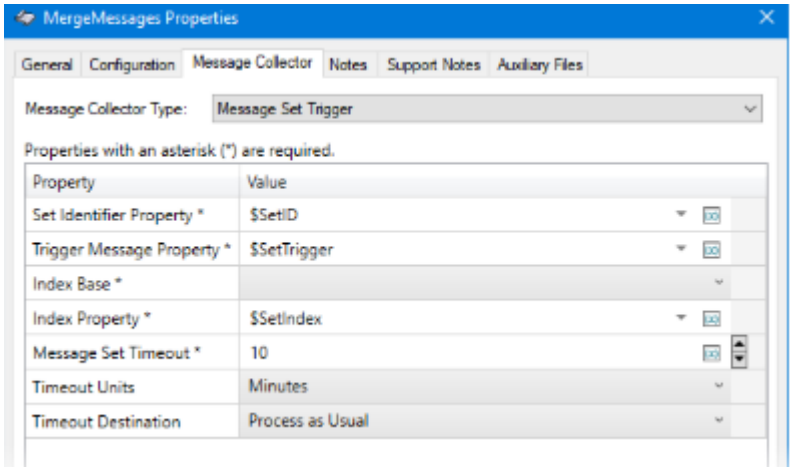


Message Set Trigger

The Message Set Trigger collector utilizes the presence of a property to mark the end of the set; the set may, therefore, contain a variable number of messages. By comparison, the Message Set Collector always contains the same number of messages in the set.

The Message Set Trigger collector type requires the following properties to be set:

- **Set Identifier Property**: the property value defines the set of which the message is a member.
- **Trigger Message Property**: the existence of this property identifies the last message in the set (the value is ignored).
- **Index Base**: the base value for the Index Property.
- **Index Property**: defines the position of the message in the set.



Message Grouping

Messages may be grouped into unordered sets (their position in the set is defined simply by order of arrival) by the following collectors

- **Property**: utilizes a property to identify the group of which the message is a member.
- **Size and/or Time**: holds messages until the total size of the group exceeds a size constraint and/or the time constraint is exceeded (measured from the time of arrival of the first member of the group).
- **Trigger Message**: holds messages until a message is received which has the Trigger property defined (any value).

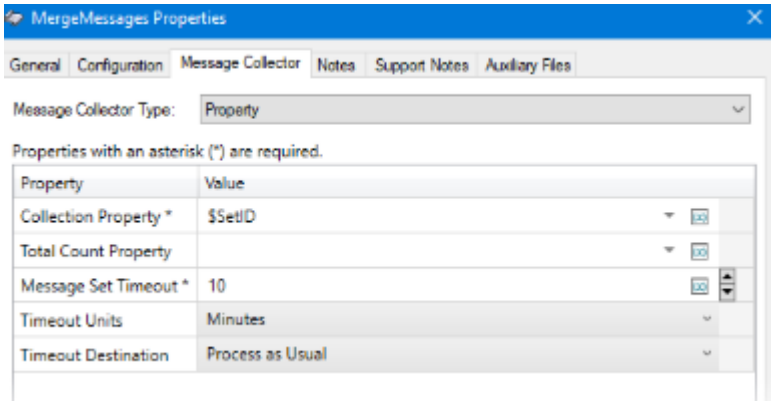
Property

The Property collector utilizes the value of a message property to define the group to which a message belongs, and releases the group when the time constraint is exceeded.

The Property collector type requires the following properties to be set:

- **Collection Property**: the name of the message property which holds the name of the group to which the message belongs.
- **Message Set Timeout**: the time, measured from the arrival of the first message in the group, to accumulate messages into the group; once the time is exceeded, the group is released to the filter.

Note that multiple groups of messages may accumulate in parallel if desired. The Collection Value Property is available on the messages in the group.

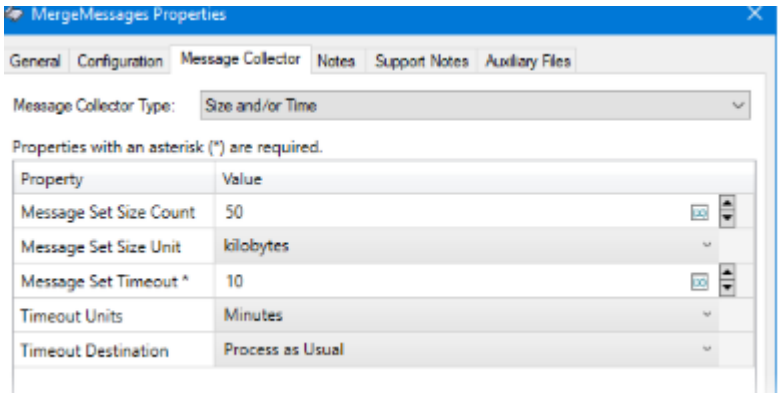


Size and/or Time

The Size and/or Time collector accumulates messages into the group until either the group exceeds the size constraint or the time constraint (note that either constraint can be disabled). All messages received by the collector are added to the group.

The Size and/or Time collector type requires the following properties to be set:

- **Message Set Size Count**: the size of the group (taken in conjunction with the units); may be set to zero (0) to ignore size constraints.
- **Message Set Size Unit**: the units to apply in conjunction with the Size parameter.
- **Message Set Timeout**: the maximum length of time to accumulate messages into the group.
- **Timeout Units**: the time unit to be applied to the timeout parameter.
- **Timeout Destination**: the destination for the group if a timeout occurs (that is, if the size constraint isn't exceeded).



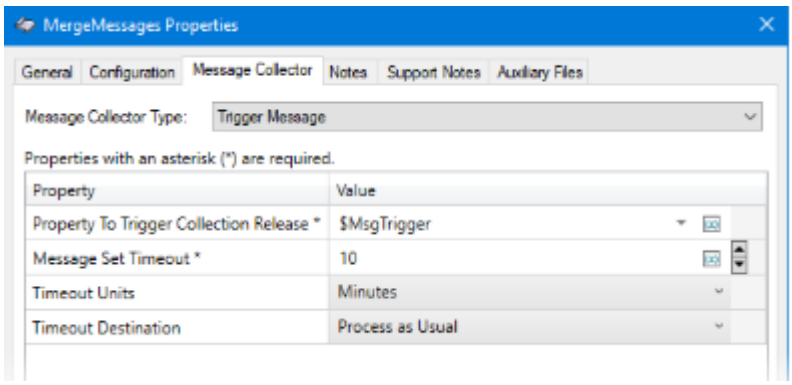
Note that it is preferable to set the timeout constraint to ensure that a message is not delayed indefinitely by the filter.

Trigger Message

The Trigger Message collector accumulates messages into a group until a message is received which has the trigger property defined (with any value).

The Trigger Message collector type requires the following properties to be set:

- **Property to trigger collection release**: the presence of this property will cause the group to be released (the value is ignored).
- **Message Set Timeout**: the length of time to allow accumulation of messages if a message with the trigger property is not received.



Message Delaying

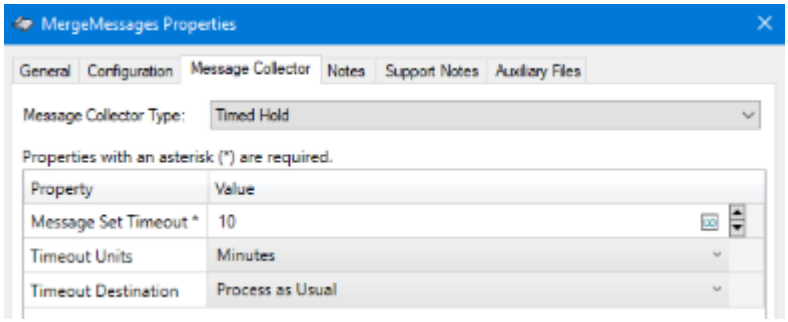
There is a single collector for delaying individual messages.

Timed Hold

The Timed Hold collector delays individual messages for the defined period without making any attempt to group messages. The defined period is measured from the time of arrival of the first message added to the group. If you need to group messages for a length of time, use the Size and/or Time collector type instead.

The Timed Hold collector type requires the following properties to be set:

- **Message Set Timeout**: the length of time that each message will be delayed for.
- **Timeout Units**: the time unit to apply to the timeout value.
- **Timeout Destination**: the destination for the messages when the timeout occurs.



Working with Collections in JavaScript

The JavaScript filter can process collections of messages. This is useful in many situations, including:

- Building the matching output collection by iterating through the members and creating a new output object for each, utilizing the output.append() method.
- Iterating through a collection of messages to build a single output message. In this scenario, the content of each message is retrieved and manipulated to create the new message body, which is then assigned to a single output message.
- Merging a query message with the corresponding response and copying selected fields and properties from one message to the other.

When using the JavaScript filter, the collection of messages is presented to the filter via the input[] array. The length of the array can be checked to determine the size of the collection.

You will use the JavaScript filter to merge a collection of messages in the extended exercise at the end of this module.

Examples

The default code for processing collected messages is similar to the following. The code uses a for loop and the length of the input collection to iterate over the messages in the collection and create an output message for each one.

```
for (let i = 0; i < input.length; i++) {
   const next = output.append(input[i]);


   // Make modifications to each message as required
}
```

Using the JavaScript filter, it is possible to mimic the behavior of the batch filter. In this example, we are creating a single output message from the first input message in the

collection. If the collection is unordered, this might not result in the expected set of message properties appearing in the output message. We then iterate over the input collection, batching all the message bodies together.

```
// Create the output message from the first input message

const next = output.append(input[0]);


let body = '';

const newline = '\r\n';


// Loop through all the input messages

for (let i = 0; i < input.length; i++) {

   // Add the message from the input to the output body

   body += input[i].text + newline;

}


// And the new message body to the output message

next.text = body;
```

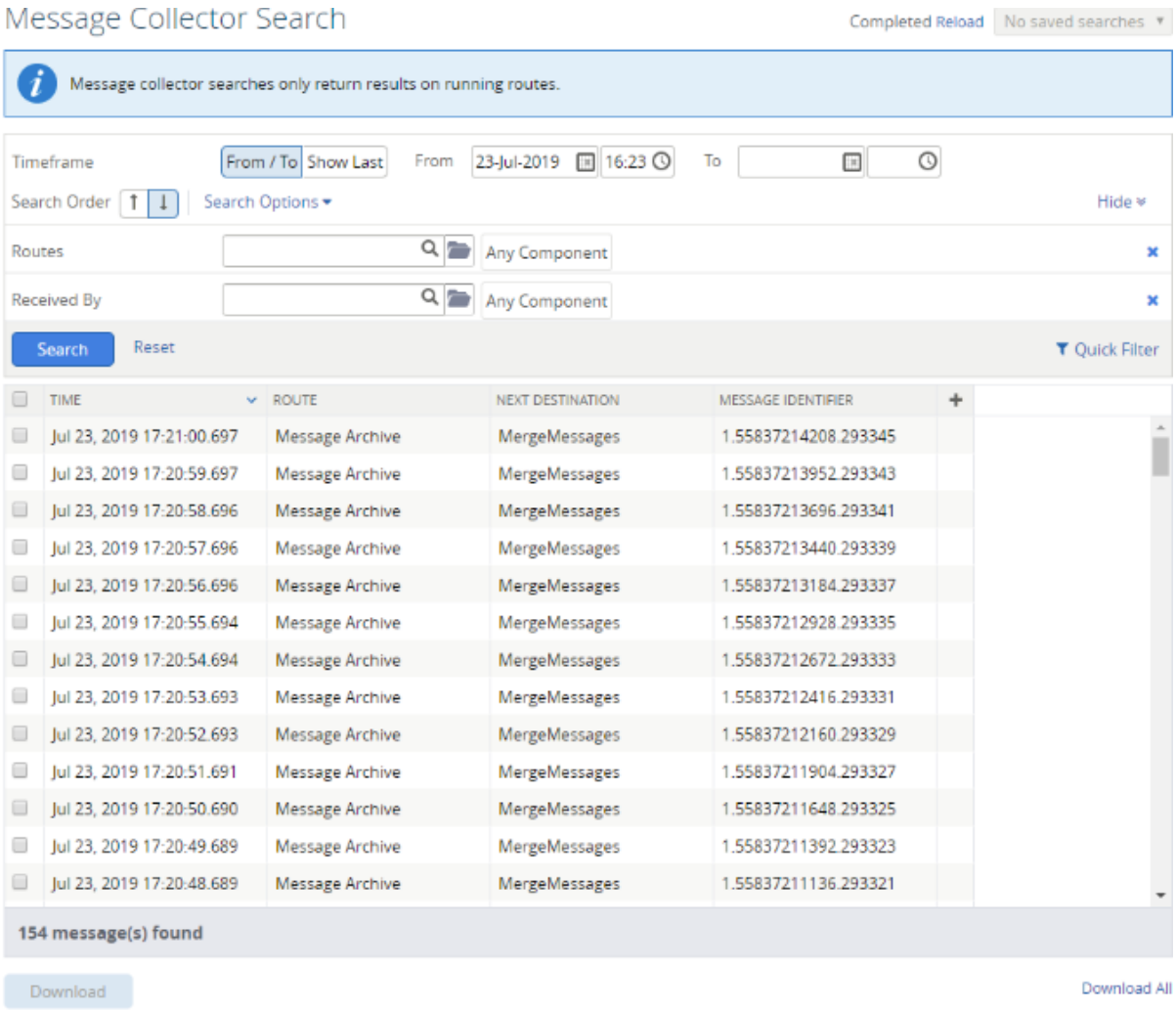
In this example, the JavaScript filter is associated with an Input Collation collector that collects a QRY^A19 query message and the associated ADR^A19 response. We only generate an output for the response message, which effectively merges the query message into the response message. The filter has been configured to copy selected fields and properties from the QRY to the ADR message.

```
// We are using an input collation collector that guarantees the order that

// messages arrive in the filter. The original query message will always be

// first, with the ADR response second.

const qry = input[0];

// Create the output message from the ADR response message.

const adr = output.append(input[1]);
```

```
// Copy content from qry to adr message.
adr.setField('QRD.QueryFormatCode', qry.getField('QRD.QueryFormatCode'));
adr.setField('QRD.QueryID', qry.getField('QRD.QueryID'));

// Copy select message properties to adr message
adr.setProperty('SiteId', qry.getProperty('SiteId'));
```

Viewing Messages in a Message Collector

It is possible to view messages currently waiting in a message collector using **Message Collector Search** in the **Management Console**. This is useful when debugging or troubleshooting the message collectors on a route. The search helps you identify messages that have been sitting on message collectors longer than expected or have not been included in a collection.



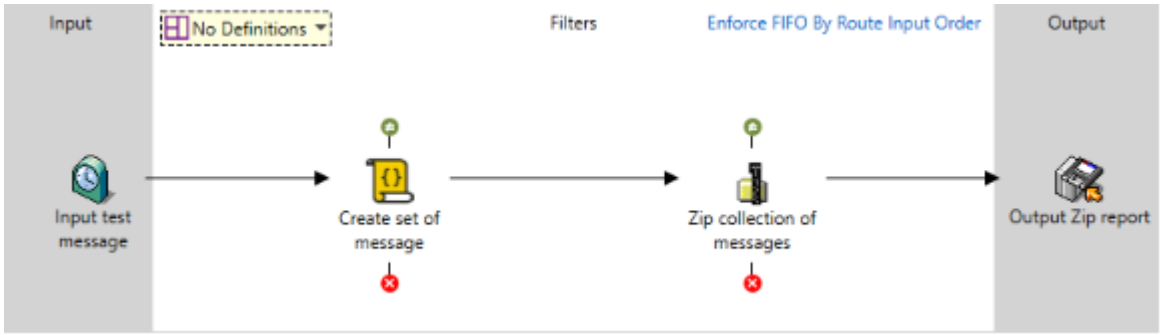
Message Collector Search

Exercise - Message Collectors

Introduction and setup

In this exercise, we will create a configuration that allows us to experiment with the various message collector types.

We start by creating a route that generates a sequence of messages. The route will group the messages into a zip archive using a variety of collector approaches. Start by creating a Timer communication point, a Directory communication point configured in output mode and the following route.



Timer communication Point

A Timer communication point is configured to load a test file containing a set of test messages at a rate of one every 30 seconds.

Configuration Property	Value
Body Template File	reports.file
Refresh Rate	30000

JavaScript

The JavaScript filter takes the input message and splits it into a number of messages (one message per row of the file). The provided test file has 30 rows, this (combined with the timer settings) will generate 60 messages per minute at the output of this filter.

Configure the JavaScript filter with the following code:

```
// Generate unique message id using incCounter method

const msgId = incCounter('Message Collector Testing');


// Split report into messages

const msg = input[0].getText('UTF8');

const msgSet = msg.split('\n');


for(let i = 0; i < msgSet.length; i++) {

  // Create the output message

  const next = output.append(input[0]);
```

```
    // Set message body to correct set item
    next.setText(msgSet[i], 'UTF8');

    // Split message content into fields
    const msgContent = msgSet[i].split('|');

    // Create a property that tells the Zip filter how the message should be named
    next.setProperty('ZipFullFilename', msgContent[0] + '-' + msgContent[2] + '.txt');

    // Set message properties to be used when testing the various collector types
    next.setProperty('MsgIndex', i);
    next.setProperty('MsgSetSize', msgSet.length);
    next.setProperty('MsgId', msgId);

    if(i == (msgSet.length - 1)) {
       next.setProperty('MsgLastItem', 'True');
    }
}
```

Each message output by this filter has several properties set. These will be used by the message collector to let us explore the various ways that messages can be grouped. The table below details the properties that are set by the filter.

Property	Description
MsgId	Identifier generated using a counter that will be unique for all messages derived from the same input message.

Property	Description
MsgIndex	The index of the message within the CSV file. A zero based index is used (this will be important later in the exercise).
MsgSetSize	The total number of messages found in the CSV file.
MsgLastItem	Set with a value of "True" on the last message to be extracted from the CSV file.

Zip / Unzip filter

The Zip / Unzip filter places each of our messages into a zip file. We will modify this filter throughout the exercise to illustrate the behavior of the various message collector types.

Configuration Property	Value
Mode	Zip

Directory communication point

The Directory communication point should be configured to write files out to a directory of your choice as follows.

Configuration Property	Value
Mode	Output
Output Directory	*Any convenient location*
Base Filename	$MsgId
Suffix	.zip

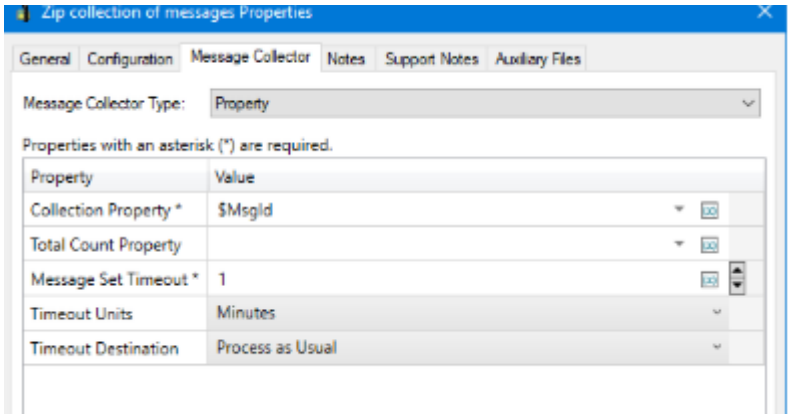
Checking the route

Check in and start all components. Open the output directory; you should see 30 Zip files added every 30 seconds. Each file corresponds with one record from the *reports.file* file used in the configuration of the Timer communication point.

The exercises on the following pages are designed to illustrate the effect of changing the Message Collector settings on the number and content of output messages.

Property collector

The first collector type we will test is the **Property** collector. This will group all messages in a set based on a message property. The property that will be used is the MsgId property. Configure the Message Collector tab of the Zip / Unzip filter as follows:



Configuration Property	Value
Collection Property	$MsgId
Message Set Timeout	1
Timeout Units	Minutes
Timeout Destination	Process as Usual

Check in, then start all components associated with the configuration.

With the configuration running, open a file manager and inspect the directory where Rhapsody is saving output messages. Notice that it takes one minute for the first zip file to appear. After that, a new item will appear every 30 seconds (this is the frequency at which the timer is generating new messages). Inspect a zip file. It should contain a set of 30 individual files (the number of records contained in each incoming file).

This delay between the timer generating the file and it appearing in the output occurs because the collector must wait for the message set timeout duration, as we have not provided details on the size of the message set.

What you should see

- After one minute, a zip file will appear. This delay matches the value of the **Message Set Timeout** property.
- An additional file is added to the directory every 30 seconds. The files will have an incrementing filename and the frequency of their appearance matches the rate at which the **Timer** communication point is injecting new messages into the route.
- Opening one of the files reveals that it contains thirty messages.
- Inspecting a file in the **Management Console** reveals that it has an associated property named MsgId whose value matches the message's filename.
- The **Message Collector Search** in the **Management Console** will show up to 60 messages waiting on the message collector.

Specifying a message set size

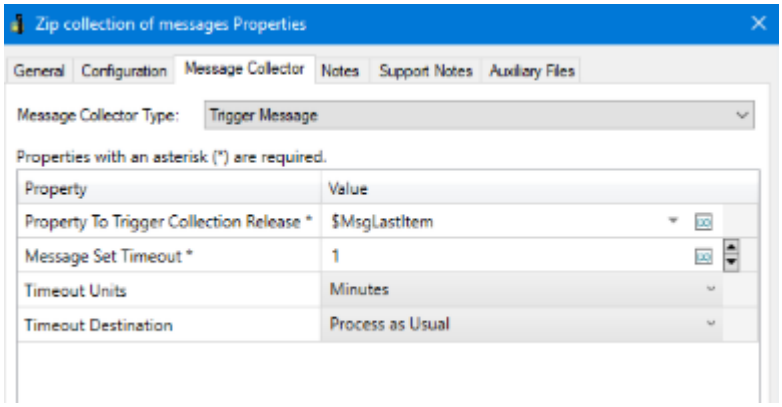
Stop the Timer communication point. Modify the collector by setting the **Total Count Property** with a value of $MsgSetSize. Check in the changes, then restart the timer communication point.

Zip messages now appear in the output directory immediately. This is because the Total Count Property causes a zip file to be generated as soon as the number of messages waiting on the zip filter matches the value set in the property.

Trigger Message Collector

We will now configure the test configuration to use the **Trigger Message** collector. This collector groups messages every time a message is received at the collector with the $MsgLastItem property set.

Stop the Timer communication point, then make the following change to the message collector on the Zip filter.



Configuration Property	Value
Property to trigger collection release	$MsgLastItem
Message Set Timeout	1
Timeout Units	Minutes
Timeout Destination	Process as Usual

Check in then start all components.

With the configuration running, a new file will appear in the output directory every 30 seconds. If we inspected a message in the management console, we would see each message moves from the input timer to the output directory almost immediately.

What you should see

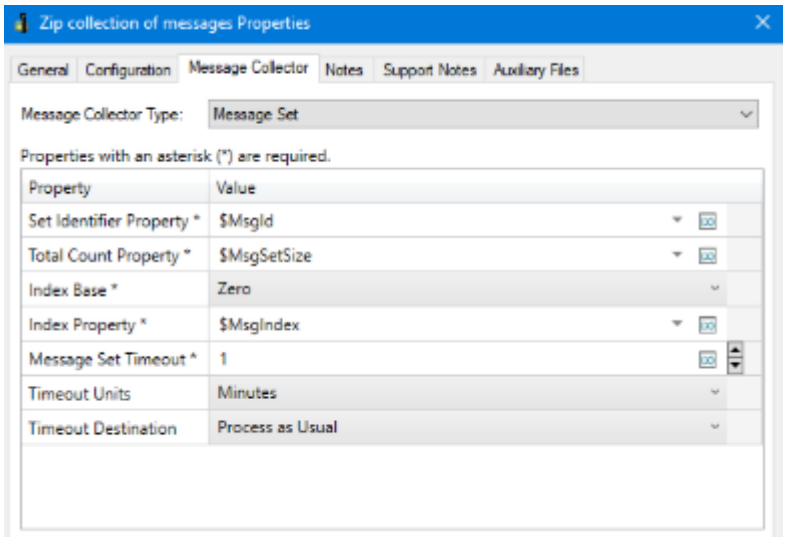
- A zip file immediately appears in the output directory.
- An additional file is added to the directory every 30 seconds.

- As Rhapsody only enforces FIFO rules at the end of the route, it is possible for messages to arrive at the message collector in a different order than when they left the JavaScript filter. In this case, it would cause the collector to group an incomplete set of messages.
- In a real-world configuration, this type of message collector should only be used when we are certain that the message containing the trigger property is only delivered to the route containing the collector after all other messages in the collection have arrived.
- Where a trigger message needed to be used in situations where message might arrive out of order, the Message Set Trigger collector could be used instead.

Message Set Collector

Next, change your configuration to a message set collector. Stop the Timer communication point, then modify the message collector created previously with a

Message Collector Type of **Message Set**. Edit the collector so that it has the following configuration:



Configuration Property	Value
Set Identifier Property	$MsgId
Total Count Property	$MsgSetSize
Index Base	Zero
Index Property	$MsgIndex
Message Set Timeout	1
Timeout Units	Minutes
Timeout Destination	Process as Usual

As before, check in then start all components.

With the configuration running, a new file will appear in the output directory every 30 seconds. Inspecting a message from the management console will show that each message moves from the input timer to the output directory almost immediately. If this collector was used on a filter where order was important (for example, a JavaScript filter or the Batch / De-batch filter), the Index Property would control the order of the messages in the set.

What you should see

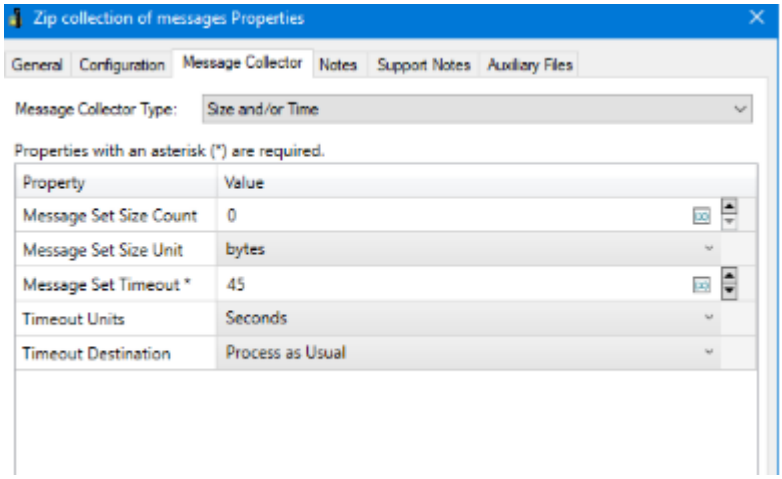
- A zip file immediately appears in the output directory.
- An additional file is added to the directory every 30 seconds.

- The **Message Collector Search** in the **Management Console** will generally show no messages (as all 30 received messages will immediately be grouped and released).

Size and/or Time Collector

Change the collector type to **Size and/or Time**. We will start by grouping solely on time.

Stop the Timer communication point, then configure the collector as follows:



Configuration Property	Value
Message Set Size Count	0 *(this disables size based collection checks)*
Message Set Size Unit	bytes
Message Set Timeout	45
Timeout Units	Seconds
Timeout Destination	Process as Usual

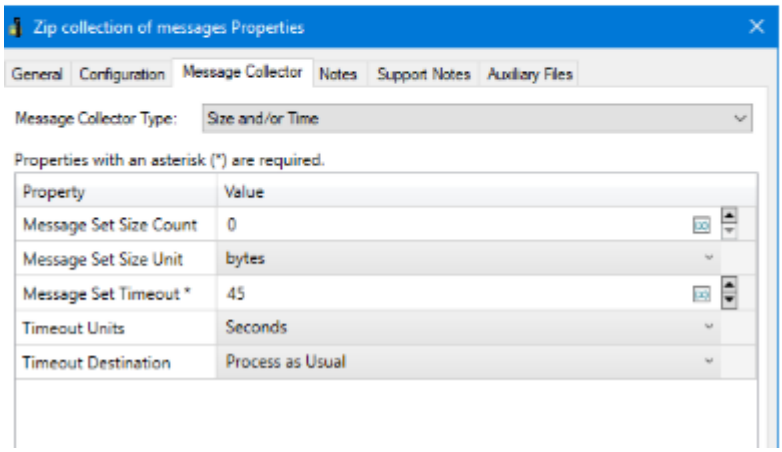
Check in then start all components.

What you should see

- A zip file appears every 1 minute in the output directory.
- Files contain either 30 or 60 messages (based on the interaction of the 30 second message generation timer and the 45 second collection timeout).
- The **Message Collector Search** in the **Management Console** will show up to 60 messages waiting on the message collector.

- **Including message size logic**
- We can also make this collector type group messages using a size value. Stop the Timer communication point and then modify the collector as follows:

Configuration Property	Value
Message Set Size Count	0 *(this disables size based collection checks)*
Message Set Size Unit	bytes
Message Set Timeout	45
Timeout Units	Seconds
Timeout Destination	Process as Usual



Check in then start all components.

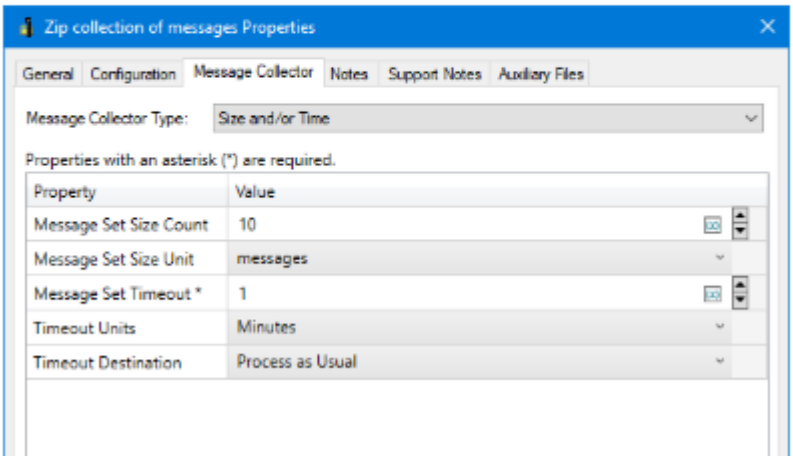
What you should see

- A zip file appears every 1 minute in the output directory.
- Files contain either 30 or 60 messages (based on the interaction of the 30 second message generation timer and the 45 second collection timeout).
- The **Message Collector Search** in the **Management Console** will show up to 60 messages waiting on the message collector.

Including message size logic

We can also make this collector type group messages using a size value. Stop the Timer communication point and then modify the collector as follows:

Configuration Property	Value
Message Set Size Count	10
Message Set Size Unit	messages
Message Set Timeout	1
Timeout Units	Minutes
Timeout Destination	Process as Usual



Check in start the configuration. Check a new output zip file to see what it contains.

What you should see

- A zip file is created every time there are 10 files in the collector.
- The **Message Collector Search** in the **Management Console** will generally show no messages (as all 30 received messages will immediately be grouped and released).

If the Message Set Size Count property was increased to a value greater than 60 (this being the number of messages that will reach the collector every minute), we would see a zip file created once per minute as the timeout settings would take over at this point. If the **Timeout Destination** was changed to a value of No-Match Connector as well, all

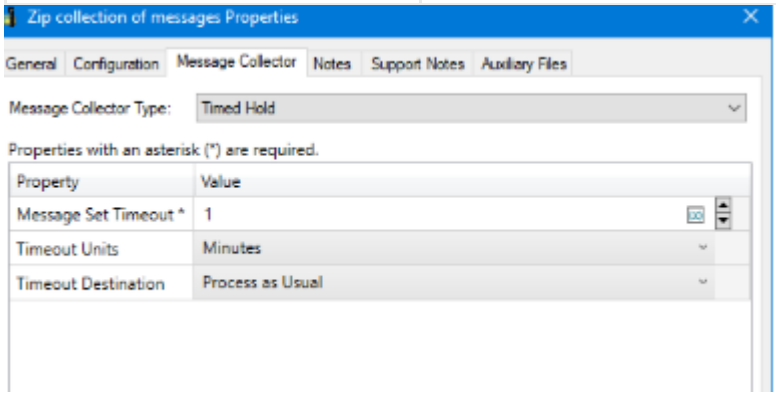
messages that had waited more than a minute on the collector would be routed to the No-Match connector.

Timed Hold Collector

Finally, we will configure a Timed Hold collector. The timed hold collector holds individual messages for the specified time period. This is different to all the collector types we have looked at so far that group multiple messages in the filter.

Stop the Timer communication point and then change the collector type to Timed Hold and configure the collector as follows:

Configuration Property	Value
Message Set Timeout	1
Timeout Units	Minutes
Timeout Destination	Process as Usual



After checking in and starting the configuration, look at the output directory. Files will start to appear in the directory one minute after the configuration is started. You will observe that 30 new zip files are added to the directory every 30 seconds. If you inspect a zip file, it will contain a single file (as this collector makes no attempt to group related messages together). The bursty nature of the file output is due to the timer generating new messages every 30 seconds and the FIFO settings on the route that prevent files from being output out of order.

What you should see

- A zip files will start to appear in the output directory after 1 minute.
- Each zip file will contain a single message.
- An additional 30 files will appear in the output directory every 30 seconds.

- The **Message Collector Search** in the **Management Console** will show up to 60 messages waiting on the message collector.

Exercise - Grouping Message from Different Sources

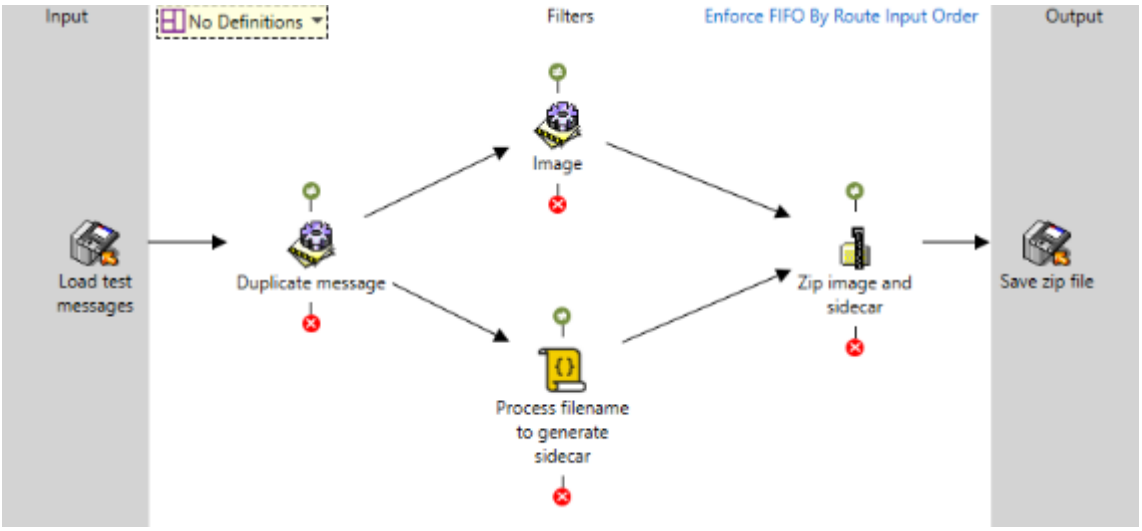
Introduction

In this exercise, we will take a set of image files containing metadata encoded into the file name and generate a Zip file containing both the original image and a sidecar file(opens in a new tab) containing the metadata from the filename.

The purpose of the exercise is to show how a message collector can be used to gather messages from several filters and group them together.

Create Route

Start by creating the following route:



Configure the Input Directory communication point to load *.jpg files from a directory of your choosing. Configure the Output Directory communication point to save the resulting files with a .zip suffix to a second directory. You may want to use the $BaseFilename message property (created by the input communication point) as the output filename.

JavaScript filter

The JavaScript filter should be configuration as follows:

```
// Create the output message

const next = output.append(input[0]);

// Split filename into array
```

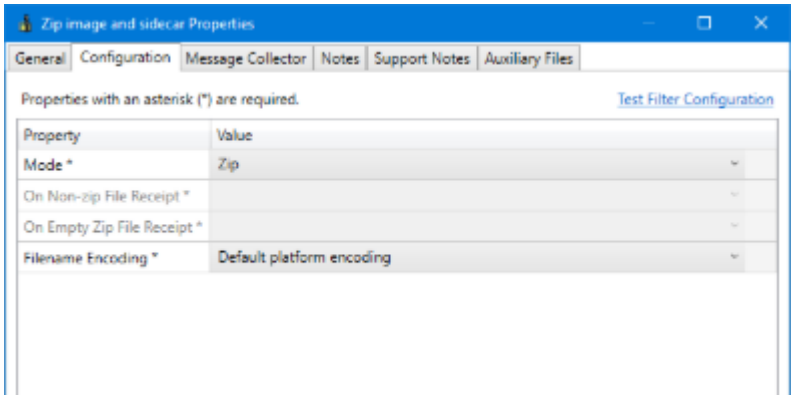
```
const fileParts = next.getProperty('BaseFilename').split('-');


if(fileParts.length < 4) {
   // Raise an error to prevent generation of the sidecar file
   next.addError('Invalid filename');
} else {
   // Generate sidecar file from filename array
   next.xml =
'<report><patient><id/></patient><order><date/><orderNumber/><service/></order><
/report>';


   next.setField('/report/patient/id', fileParts[0]);
   next.setField('/report/order/date', fileParts[1]);
   next.setField('/report/order/orderNumber', fileParts[2]);
   next.setField('/report/order/service', fileParts[3]);


   // Set message properties required by zip filter
   next.setProperty('ZipFullFilename', 'meta.xml');
}
```

The JavaScript filer has been configured so that files with filenames that do not match the expected pattern are sent to the error queue. In a production environment, these events should be detected and managed without using the error queue. Error Handing is discussed in more detail later in this module.

Zip / Unzip filter

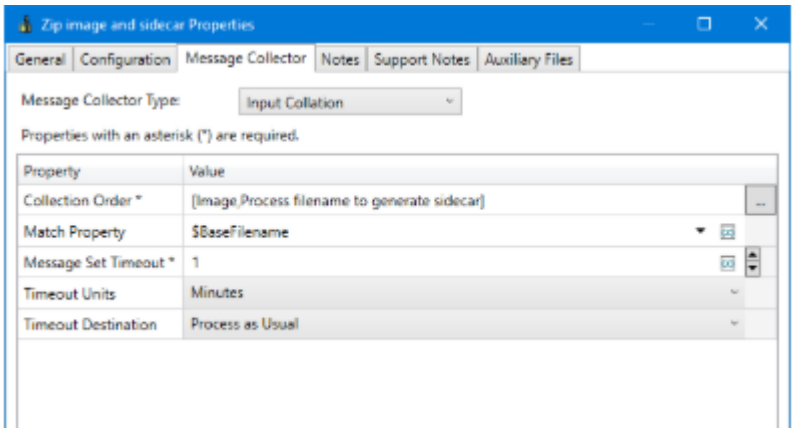
Configure the Zip / Unzip filter as follows.

Set the **Mode** for the filter to Zip.



As we are using a zip filter, where input message order does not matter, the order of values in the **Collector Order** doesn't matter. However, entries for the two preceding filters must be present, or the collector will generate an error. If the collector was associated with a filter where order mattered (such as the JavaScript Filter or Batch / De-batch Filter), the value of this property would be important.

Set the **Match Property** to $BaseFilename. This property contains the base portion of the input filename. We can assume that it is unique for this exercise (if this property wasn't unique, or was missing from the collector configuration, messages could be incorrectly matched).



Set the **Message Set Timeout** to 1 minute. This ensures that any timeouts that occur resolve themselves quickly (if we used the default value, we would need to wait ten minutes to see any failed matches appear).

Testing the Configuration

Download the set of test messages

Copy each file - except *Error.jpg* - to the input directory. Observe how all files are processed by Rhapsody almost instantly. Open each of the zip files in the output directory and see what files it contains.

Now copy the *error.jpg* file to the input directory. The filename of this file does not meet the requirements for generating the sidecar metadata file and is sent to the error queue. Notice that it takes one minute for this file to be processed (this is the length of the timeout on the collector) and the output zip file does not include the sidecar file. During this time you will be able to view the file from the **Message Collector Search** in the **Management Console**.

If you copied a second file to the input directly shortly after an error file was uploaded for processing, you would need to wait for the error file to timeout inside the collector before the second file appeared at the output. This is due to FIFO rules requiring the error file (that arrived in Rhapsody first) to be output first.

Answer & Explanation

If using a Property collector or a Message Set collector, we would need to provide the total number of expected messages to the collector so that it is able to determine when the set is complete. This is done with the **Total Count** property.

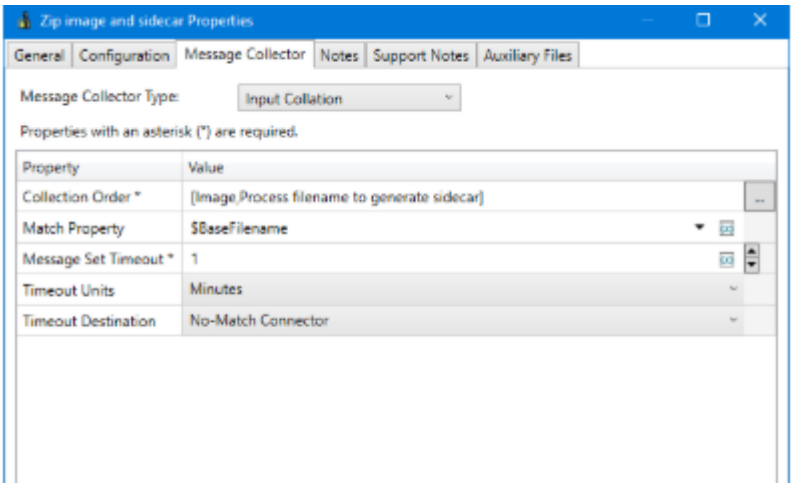
The following properties are not required:

- The Index property is only needed on collectors that order the set of collected messages.
- The Trigger message property is only needed when using a collector that uses a trigger message to indicate set completion.
- The Messages Set Size Count is only used for Size and/or Time type collectors.

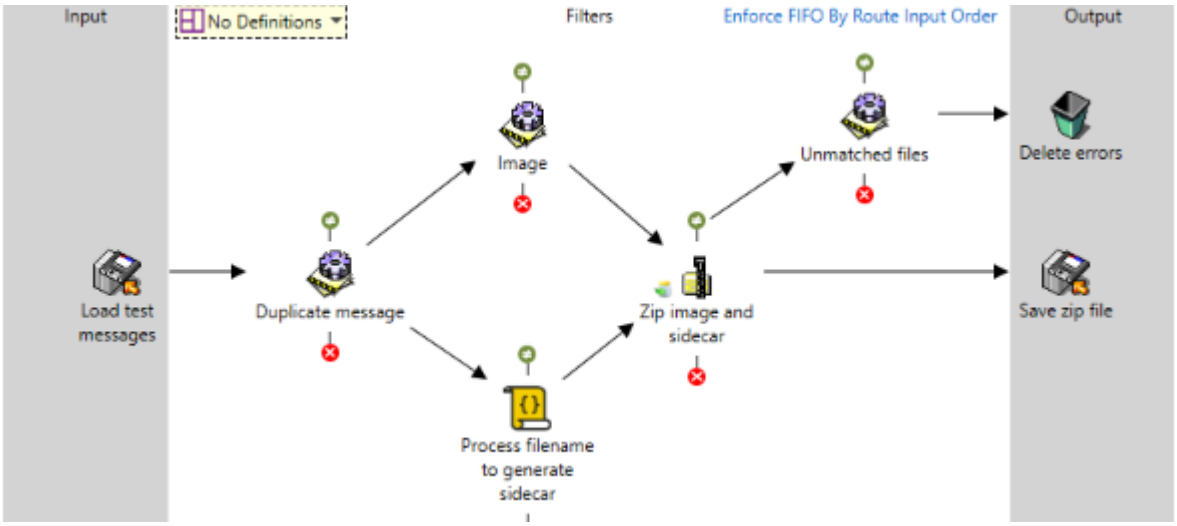
- # Extension - Doing something different with unmatched files

- In most real solutions, having messages process as normal after a timeout on a collector is dangerous and can lead to broken configurations or messages. In this extension, we will change the behavior of the collector so that messages that do not correctly generate a sidecar file do not generate a Zip file.

Modify the **Timeout Destination** from the **Message Collector** tab of the Zip / Unzip Filter to a value of **No-Match Connector**.



We now need to modify our route to process any messages that arrive on the no-match connector.



For this exercise, we have sent the unmatched files to a sink. In a real-world solution, you would need to decide the correct action. This might involve sending an error report to the sending and/or destination systems, sending an error notification to an administrator, or inserting default content into the message to make it valid.

Snapshot Filter

Snapshots

A message snapshot can be used to save the body of a message, then at a later point in the configuration recall it. This can be used where the previous state of a message needs to be retrieved. For example, a message is sent to an external system for processing, with the response being used to determine how further processing of the original message will be performed. In older versions of Rhapsody, recalling an existing message state would require the use of the Input Collation Message Collector.

Saving snapshots

A message snapshot can be taken at any time with the Snapshot Save filter. This filter saves a reference to the current message in the rhapsody:SnapshotMessageId and rhapsody:SnapshotBodyEncoding message properties.

The values recorded in the rhapsody:SnapshotMessageId and rhapsody:SnapshotBodyEncoding message properties will be replaced by each subsequent instance of the filter. It is possible to record the state of a message at multiple points by repeatedly calling the **Snapshot Save**, and saving the set of properties. To reload a particular snapshot, the appropriate state is selected and saved back to the message property before loading to obtain the appropriate snapshot state.

Loading Snapshots

An existing snapshot can be loaded using either the Snapshot Load filter or from a JavaScript filter.

Snapshot Load Filter

The Snapshot Load filter uses the properties added to a message by the Snapshot Save filter to restore the message body to the point when the snapshot was taken. The filter only loads the message body. Properties are not altered by the filter.

JavaScript Filter

There are two JavaScript functions for loading a snapshot. In both cases, the snapshot is loaded as a read-only message object. This is useful when elements need to be taken from the current and snapshot states of the message, compared and then combined into a single output message.

Function	Description
loadMessageSnapshot()	Loads the snapshot of the message which was saved with the Snapshot Save filter and returns it as a ROMessage object. This object contains the same message body and properties that the message had when it was saved.
loadMessageSnapshotForId(snapshotMessageId)	Loads the message snapshot with the specified message ID saved with the Snapshot Save filter and returns it as a ROMessage object. This object contains the same message body and message properties that the message had when it was saved.

In both cases, an error will be raised if you attempt to load a snapshot message after the Snapshot Save filter that created the snapshot has been deleted, or if you attempt to load a snapshot in a different locker from the one where it was saved.

Error Handling

Overview

The expectation is that all messages sent by a source system are delivered to the appropriate destination(s). All configured data flows should also handle expected exceptions and errors. If there is an unexpected error that is not handled by the configuration, the message will be sent to the Error Queue. When this occurs, the error needs to be analyzed to determine the root cause. Once the root cause is identified, there are several options that can be taken to correct the error. All messages in the Error Queue should be removed in a timely fashion for multiple reasons.

Notifications should be configured to inform personnel when any message is in the Error Queue so the message(s) can be quickly corrected and delivered to recipients. The expectation is that the Error Queue does not contain any messages and that all

messages reach their expected destinations or are filtered based upon pre-defined business requirements.

Principles

The following principles should be kept in mind when incorporating error management into your Rhapsody solution:

- Ensure error management is integral to your route design and not tacked on at the end of the project because it is something you are supposed to have.
- Test your configuration at regular intervals so you know that your route works at each stage of the build before moving on to the next. Ensure configuration errors are routed to a location that is checked regularly so that they do not get lost or overlooked.
- Include a route pathway in your solution for unknown messages. Check to find out why these messages are unknown and whether they can be corrected at source so that they do not become a recurring problem.
- Ensure that messages are placed into the Error Queue only in truly exceptional circumstances; everything else should be anticipated and managed.

Error Handling Mechanisms

Rhapsody provides a number of mechanisms for managing errors and/or recovering from the failure of an external system. These include:

- Sending errors to the Error Queue, a component of the Rhapsody **Management Console**. There, messages can be viewed and potentially edited before being sent back into the route for continued processing. This is the default destination for error messages.
- Using the Error Connector associated with all filters to selectively route errors occurring on different components of your route to a convenient location for regular checking and correction.
- Using the Route Error Handler to capture all errors and route them together to the same location for further scrutiny.

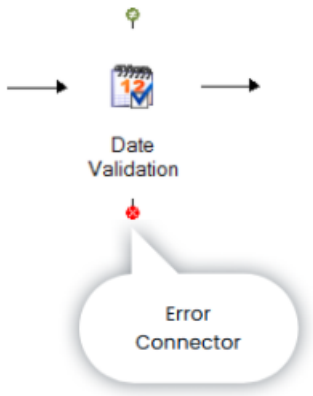
The Error Queue

All filters include an Error Connector which acts as a destination for any messages that trigger an error in the component.

If an error occurs while processing a message though a filter, Rhapsody aborts the filter and sends then message received by the filter to the Error Connector. Some filters (such

as the JavaScript filter or the Message Validation filter) are also able to direct the output of the filter to the Error Connector.

If no pathway has been configured from the Error Connector, messages reaching it will automatically be sent to the Error Queue for further management in the Management Console. Error messages can alternatively be routed to another component for further processing.



Error Queue Message Management

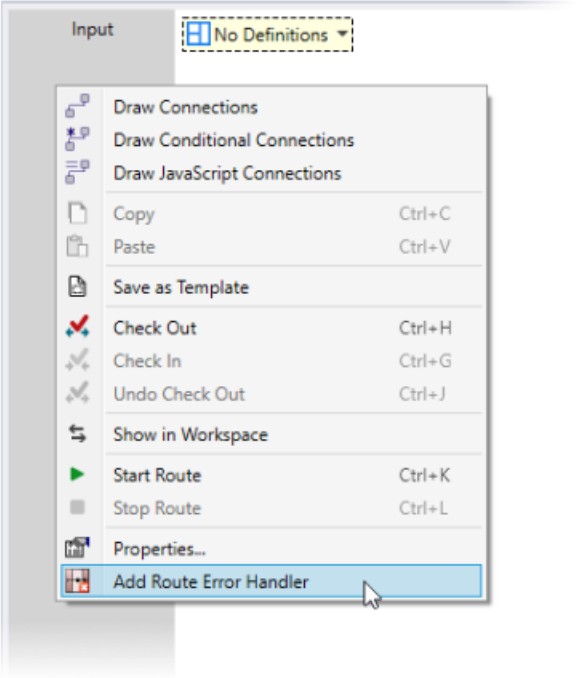
In a production system, the Error Queue should be reviewed on a regular basis and messages removed from it once a reason for the failure has been identified and an appropriate corrective action taken.

A Rhapsody system with many messages on the Error Queue is harder to manage, as it becomes very difficult to know what issues may exist within the solution and which ones have been resolved.

The Route Error Handler

The Route Error Handler is used to redirect all messages that would otherwise be sent to the Error Queue (any filter that does not have a connection from its error connector) to a specific point within a route. This is useful when there is a reasonable amount of processing occurring on a single route, or where a common set of actions needs to be performed on any message that fails within the Route. For example, if all messages received from an external system must be acknowledged before the next one is received, even if an error occurs during processing.

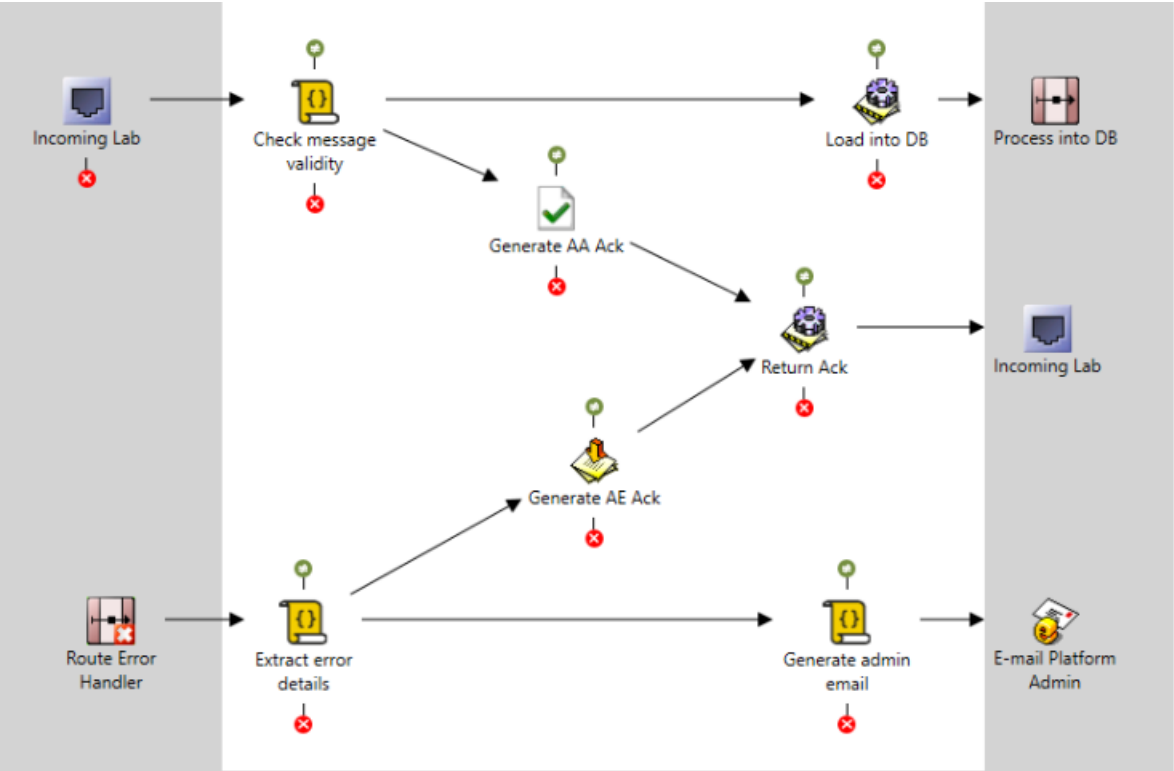
The Route Error Handler is added to a route by right-clicking on the route and selecting the **Add Route Error Handler** option.



Adding the Route Error Handler

Once the Route Error Handler has been added to a route, it behaves like an input communication point. A typical configuration for the handler will forward a copy of the failed message to a system administrator for review, while also performing some further processing on the message.

In the example below, a copy of the message is sent to a platform administrator via email, as well as being used to generate an acknowledgement. The **Content Population** filter is being used to generate the acknowledgement message, as the error may have been caused by a message that could not be parsed on receipt from the external system.



Example route illustrating the use of the Route Error Handler

A message can only be sent to a Route Error Handler once. If an error occurs in a filter that is processing a message already sent to the Route Error Handler, the message will be sent to the Error Queue. For this reason, it is important that processing of failed messages makes no assumptions about message content.

Inspecting Message Errors in JavaScript

The JavaScript filter is able to inspect errors recorded against a message. This could be used when the details of a message error need to be recorded in an Error Acknowledgement (NACK) message or in an email sent to a system administrator.

The getErrors() function returns an array containing all the errors associated with the message. This lets you retrieve all the nested errors associated with an error event. These could be used to construct a notification message or sort messages based on the type of errors detected.

The following example illustrates how the errors details could be added to a message property named ErrorMessage.

```
// Create the output message

const next = output.append(input[0]);


let errorText = '';
```

```
const errors = next.getErrors();


// Get all errors associated with message
if(errors != null && errors.length > 0) {
   for (let cErr = 0; cErr < errors.length; cErr++) {
      errorText = errorText + errors[cErr] + '\r\n';
   }
}


next.setProperty('ErrorMessage', errorText);
```

Adding an Error to a Message

Where you have a need to record an error against a message from within the JavaScript filter, the addError() function can be used. After adding an error, the message will finish processing in the filter before being sent to the error connector.

```
if(!isValid) {
   next.addError('Message is not valid');
}
```

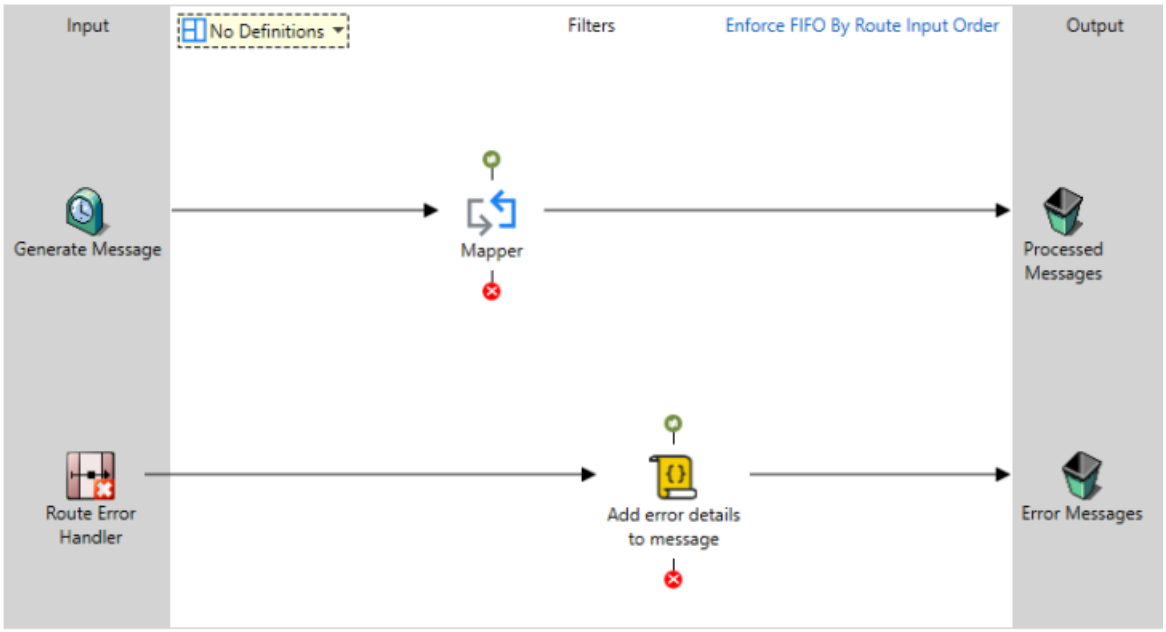
Exercise - Error Handling

Introduction

The purpose of this exercise is to illustrate Rhapsody's Route Error Handling feature.

Create test Route

Assemble a route using the following configuration options:



Error Handler route

1. Create the 'Generate Messages' Timer Communication Point.

The Body Template File property can be left blank. This Communication Point will be used to input messages into the route which are deliberately designed to fail processing on the Mapper filter. When this happens, the error messages will be picked up and routed by the Route Error Handler.

2. 2

Create the "Processed Messages' Sink Communication Point to complete the message flow in the route.

3. 3

Add a Mapper filter and associate it with a mapper definition file. It does not matter which definition file is used; messages arriving at this filter are intended to fail processing.

4. 4

Add the Route Error Handler to the route. Note that it must be connected to a filter and not directly to an output Communication Point.

5. 5

Add a JavaScript filter and configure it using the following code:

```
// Create the output message

const next = output.append(input[0]);
```

```
const errors = next.getErrors();

let errorText = '';


// Get all errors associated with message

if (errors != null && errors.length > 0) {

  for (let cErr = 0; cErr < errors.length; cErr++) {

    errorText = errorText + errors[cErr] + '\r\n';

  }

}


next.setProperty('ErrorMessage', errorText);
```

This will create a new message property called ErrorMessage containing a summary of the errors that have occurred to the message.

1. 6

Add a Sink Communication Point to receive the error messages generated by this route.

Sending error messages to a Sink Communication Point effectively causes them to be ignored. In a real configuration, you would likely use an Email or Directory Communication Point to either notify an administrator or trigger a monitoring system.

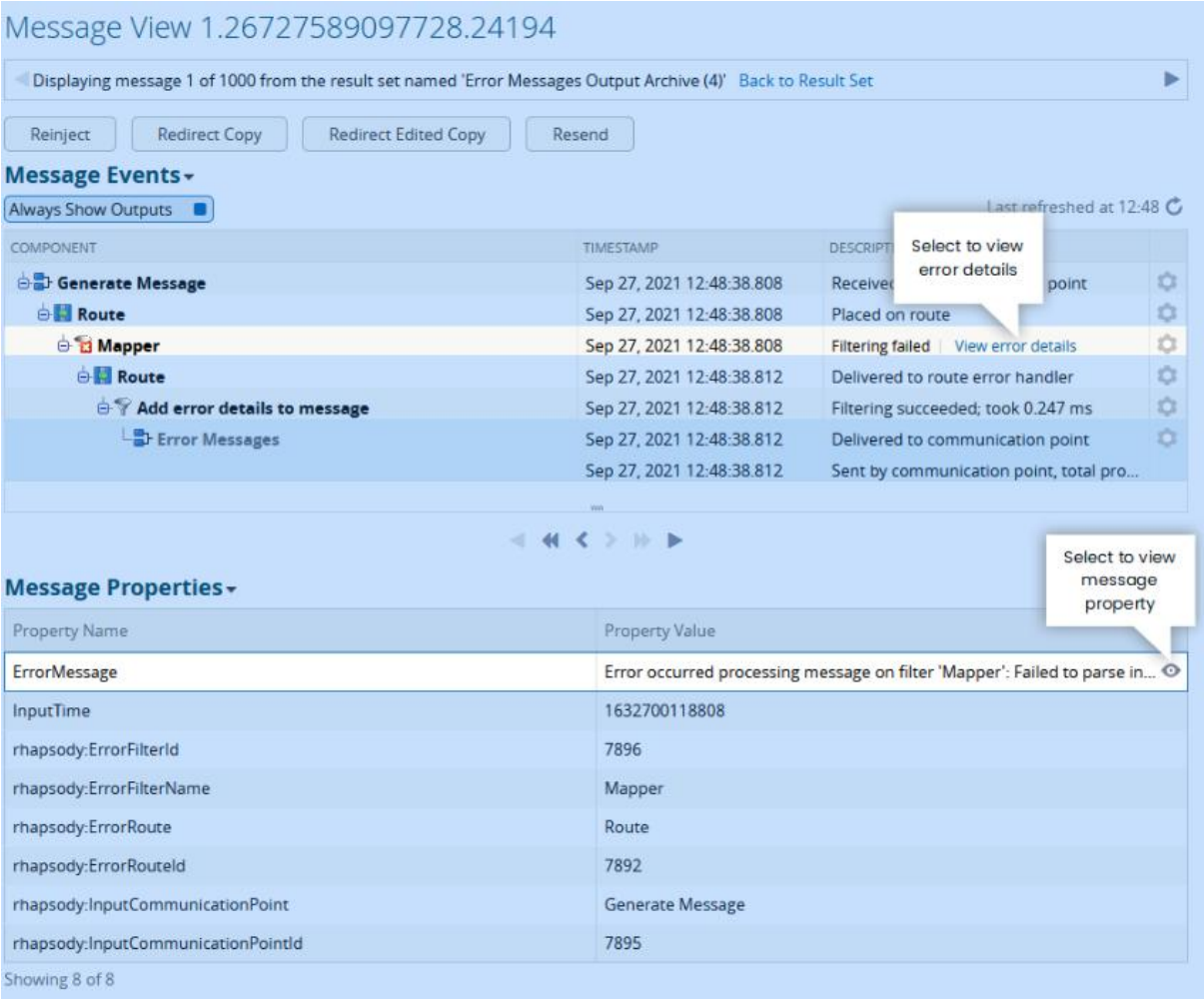
Check in and start all components.

Testing the Route

With all components started, open the **Management Console** and select **Communication Points** from the Monitoring menu. You should see:

- No messages reach the **Processed Messages** Sink Communication Point.
- The count of messages on the **Error Messages** Sink Communication Point rises as long as the timer Communication Point is running.
- Select the **Error Messages** Communication Point and, from its Details screen, open its Output Archive. Select the most recent message from this archive.

- Select the **View Error Details** link associated with the Mapper filter to see the error that occurred while attempting to map the message.
- With the Error Messages output Communication Point selected, find the **ErrorMessage** message property and select the view icon to the right of the property to see the full content of this property. The value of this property will be a description of the processing error that occurred on the route.



Testing Filters and Conditional Connectors

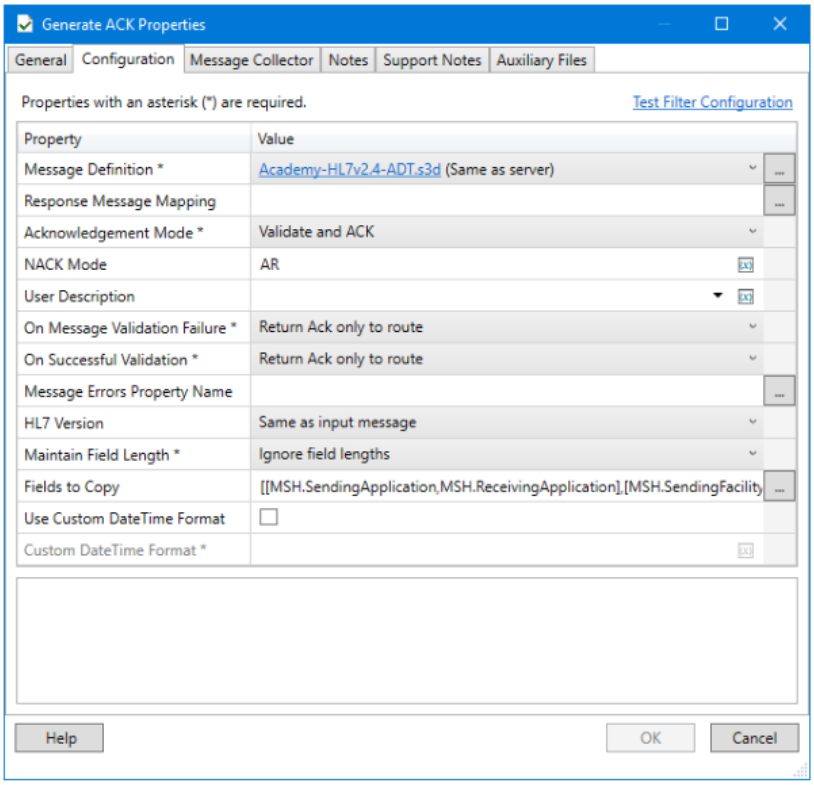
Introduction

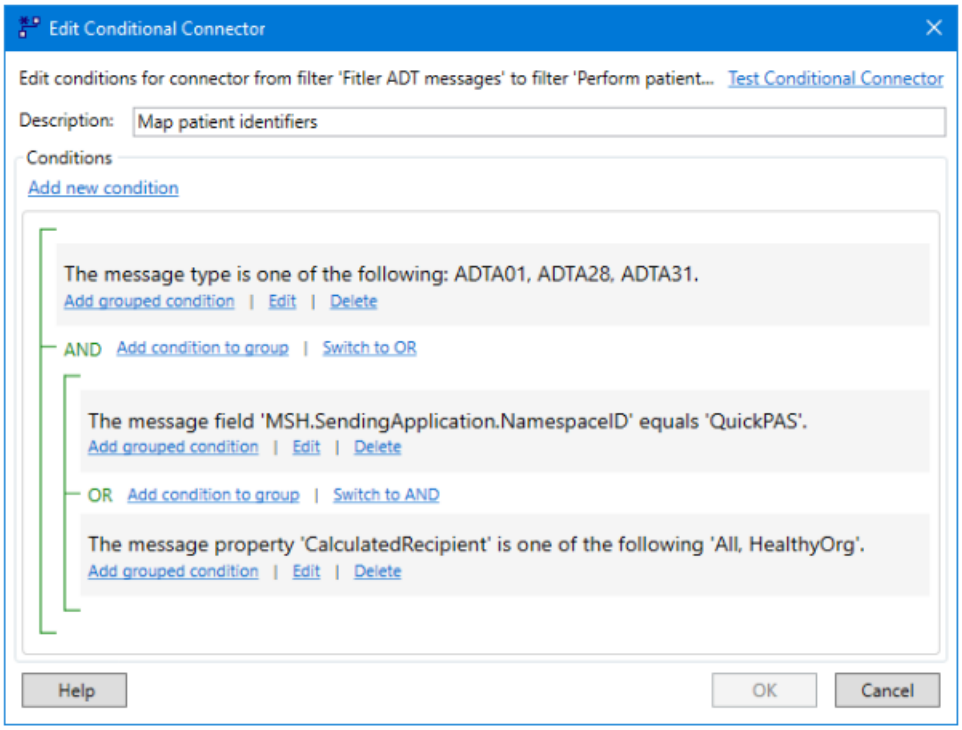
As a Rhapsody configuration becomes more complex, the ability to test individual elements of the configuration in isolation becomes more important. The primary tool for testing filters and conditional connectors in Rhapsody is the **Test Filter** and **Test**

Conditional Connector configuration. This allows these components to be unit tested from within the Rhapsody IDE. This in turn significantly reduces the time taken to develop any non-trivial filter to an error free state.

This module will cover the process of adding test messages to a filter or conditional connector and running the tests to confirm that the component is working as intended.

The testing window is displayed by selecting the **Test Filter Configuration** link from its **Configuration** tab, or by selecting **Test Conditional Connector** in a conditional connector's **Properties** window.

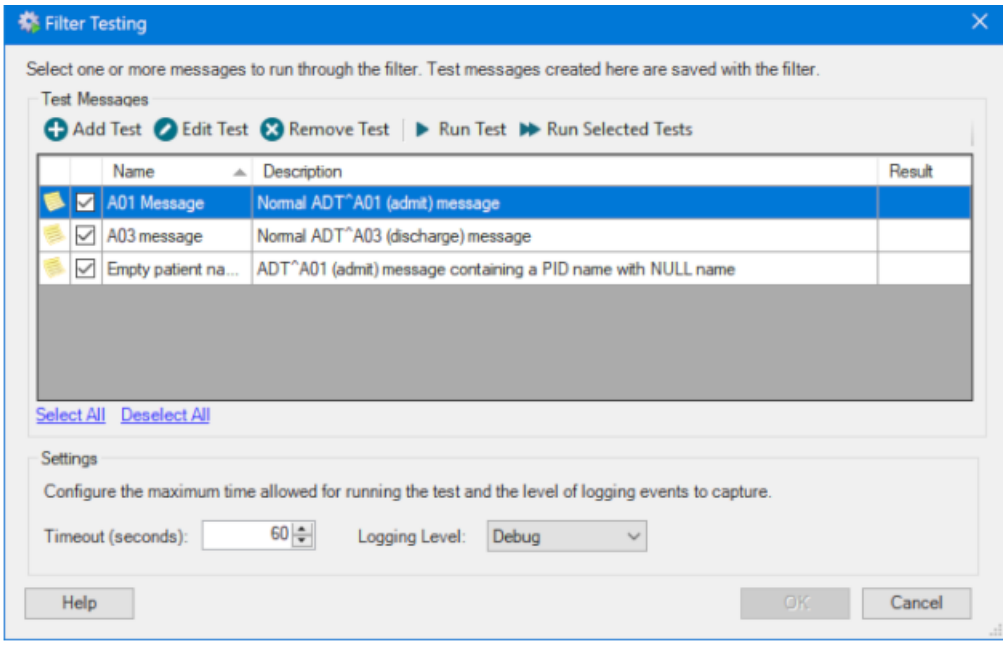




Conditional Connector

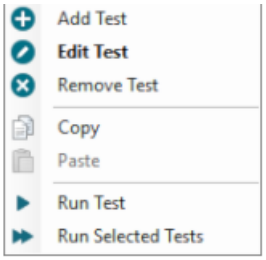
Filter Testing

Selecting either of the above displays the **Filter Testing** or **Conditional Connector Testing** screen where the component's associated test messages are managed and run. Both screens share the same options and layout.



Sort the list of test messages by selecting the **Name**, **Description** or **Result** headers, and again to reverse the sort. Individual tests can be selected (or deselected) by selecting their corresponding check-boxes. Select all tests, or clear the selection of all tests, by using the links at the bottom of the table.

Right-clicking on a test brings up a context menu that provides quick access to a number of actions, such as copy/paste, that can be performed against a test message. These are useful when many similar tests need to be created.



Settings

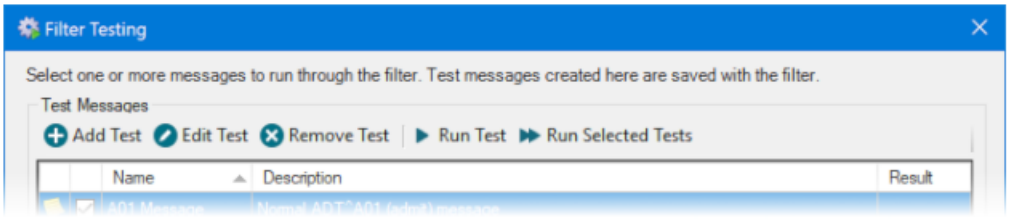
The **Settings** options at the bottom of the screen are common to both filter and conditional connector testing:

- **Timeout** - specify the maximum time (in seconds) for running the tests. This prevents tests from 'hanging' when a configuration unexpectedly causes a test to run indefinitely.
- **Logging Level** - specify the log level for errors to be reported at during the running of the test. By default, this is set to Debug.

The Test Editor

In order to test a filter or conditional connector, one or more test messages must be added to the component's testing window. The test messages should cover the range of input types the component is intended to handle and should include messages that are expected to pass along with ones that will fail for a variety of reasons.

A new test message is added by selecting the **Add Test** button. An existing test message is edited by highlighting it, then selecting the **Edit Test** button, or by double-clicking its entry in the list.

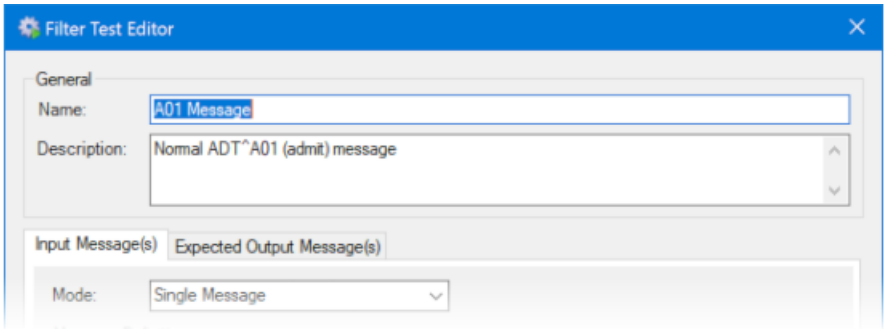


The screen for adding and editing a test message is the same.

General Properties

The **Filter Test Editor** and **Connector Test Editor** share a common set of properties laid out in a similar way.

Each test message or message set requires a unique **Name** and a **Description** that clearly identifies what aspect of the filter's configuration is being tested. This information is useful when reviewing the configuration and test results.



The remainder of this screen is divided into two tabs: **Input Message(s)** and **Expected Output Message(s)**.

Input Message(s)

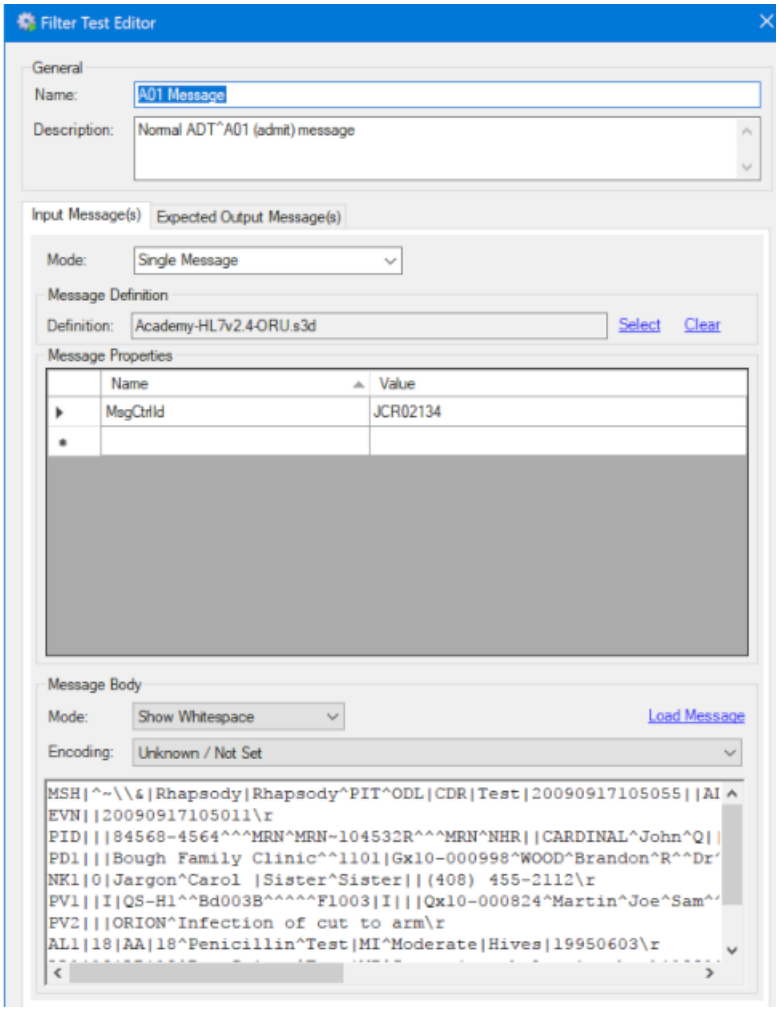
The **Input Message(s)** tab on a filter or connector's Test Editor screen configures the message or messages that make up this test message set.

When testing a filter, change the **Mode** to test either a single message or a collection of messages (when the message collector is being used with the filter). Testing of message collections will be covered later in this lesson. Conditional connector tests can only be performed against a single message.

The properties of a test message are:

1. **Definition** - (optional) can be populated with any definition available in the current Rhapsody Locker. If the filter attempts to parse a message where a definition has not been provided, the route definition will be used - which is appropriate where the filter processes messages using this definition.
2. **Message Properties** - manually add any message properties required for the filter to correctly process this test message.
3. **Message Body**- select the Load Message link to load a test message from disk. Its display options are:
    - **Mode** - The display mode of the message.

- **Encoding** - The character encoding of the message to be tested. If Unknown/Not Set is selected, no encoding will be applied to messages sent to the filter for testing.



Expected Output Message(s)

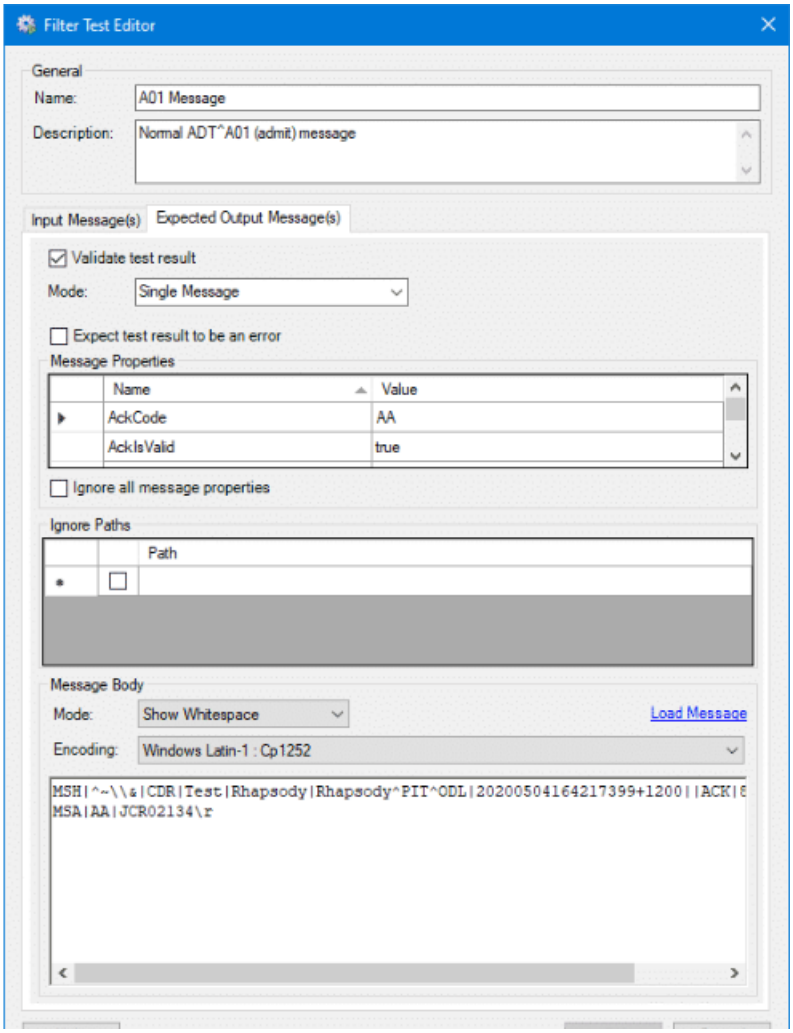
The **Expected Output Message(s)** tab on a filter or connector's Test Editor screen configures the expected output from the filter or conditional connector for a test message. Selecting the **Validate test result** check box enables output validation for the test.

When testing a filter, change the **Mode** to test either a single message or a collection of messages (for example, a De-batch filter or a JavaScript filter that calls output.append() multiple times). The message properties and body content for the expected output message are shown below.

The properties of an expected output message following a test in Single Message mode are:

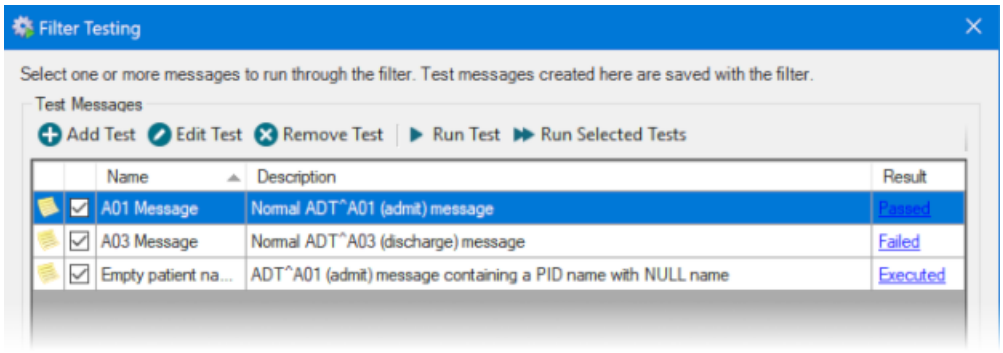
1. **Mode: Single Message** - one output message will be generated by the filter.

2. **Expect test result to be an error** - ensure the result shows as a 'Pass' when an error occurs during the processing of the test message.
3. **Message Properties** - manually add or edit message properties. These will be used during the validation of the output messages.
   Adding or editing message properties is only possible if Ignore all message properties is not selected.
4. **Ignore Paths** - add or edit the message paths whose contents are to be ignored during output message validation. This option is used for HL7 messages in HL7. Fields that are expected to change every time a test is run (for example, the MSH.DateTimeOfMessage and MSH.MessageControlID fields when generating an acknowledgement) can be set to be ignored when validating the output message. HAPI paths are used to specify the fields to ignore.
5. **Message Body**- the contents of the message body will be used during the validation of the test message.
   - **Mode** - The display mode of the test message.
   - **Encoding** - The character encoding of the output message.



Run Tests

A filter or conditional connector test is run by either selecting it then selecting Run Test, or selecting the checkbox alongside one or more tests then selecting Run Selected Tests.



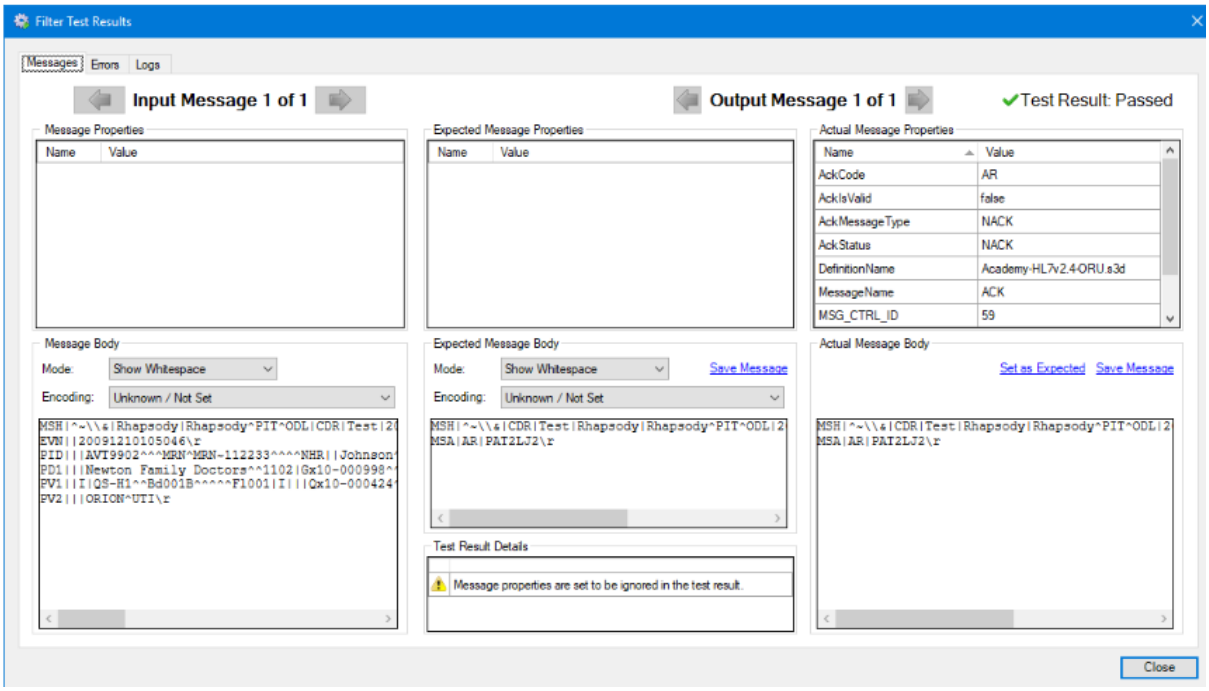
Running the test executes the component's configuration against the test message(s) and displays the result in the rightmost column. The possible outcomes are:

- **Passed** - the test has executed successfully, with the result passing validation. In other words, all message properties and content for the actual and expected messages matched.
- **Failed** - the test has executed successfully but with the result failing validation. In other words, at least one message property and/or the content of the actual and expected messages did not match.
- **Executed** - the test has executed successfully. Output messages, however, were not validated.
- **Error** - one or more fatal errors have occurred while running the test, preventing it from successfully executing.
- **Skipped** - the test did not run, usually because no input messages had been selected when in testing in Message Collection mode.
- **Invalid** - there are two causes of an invalid result:
    - The filter does not support filter testing. Examples include the Duplicate Message Detection and Hold Queue filters.
    - The filter is connecting to a non-standard database using custom database drivers. This could apply to the Database Look Up and Database Message Extraction filters.

Select the Result link for an individual test to either review the test parameters or view the reason(s) why it failed. If this column is empty (blank), the test has yet to be run. Changing a test's parameters automatically clears its corresponding Result field.

Test Results - PASS

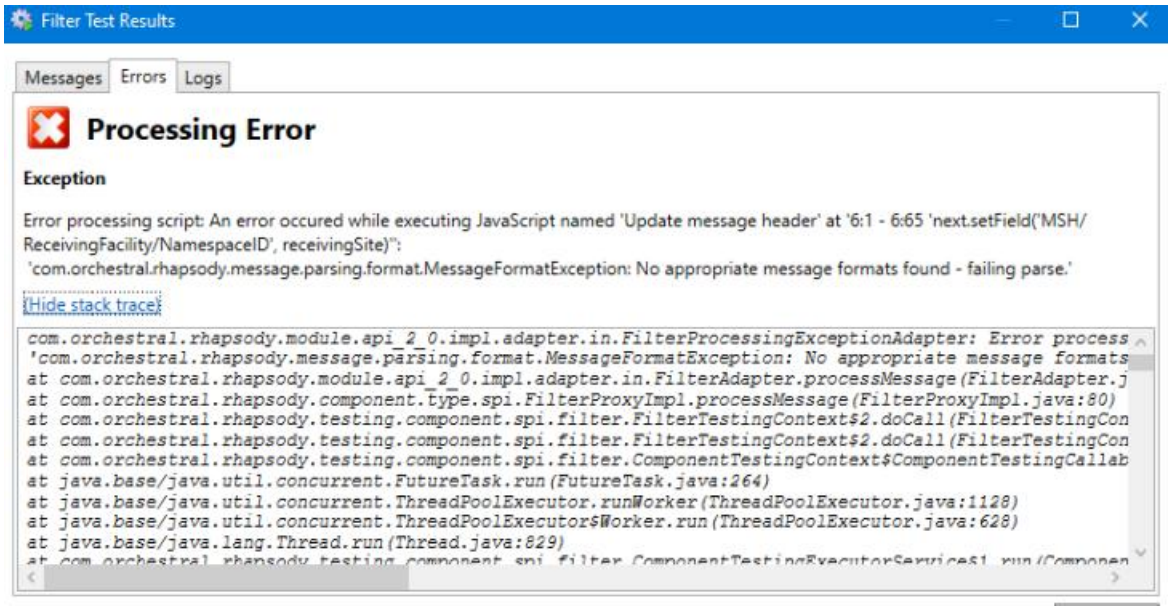
A successful result following the testing of an HL7 Acknowledgement Generation filter is shown below, with the expected message matching the actual result. Note in this case that the message properties have been ignored for this test; see the **Test Result Details** at the bottom of the middle column. If message properties had not been ignored, the test would have failed because the actual output message has a number of associated properties that have no match in the expected message properties panel.



The **Actual Output Body** panel of the search results screen includes a **Set as Expected link**. Selecting this link sets the actual output message as the expected output message. This is an effective way of quickly generating the expected test messages for positive test cases.

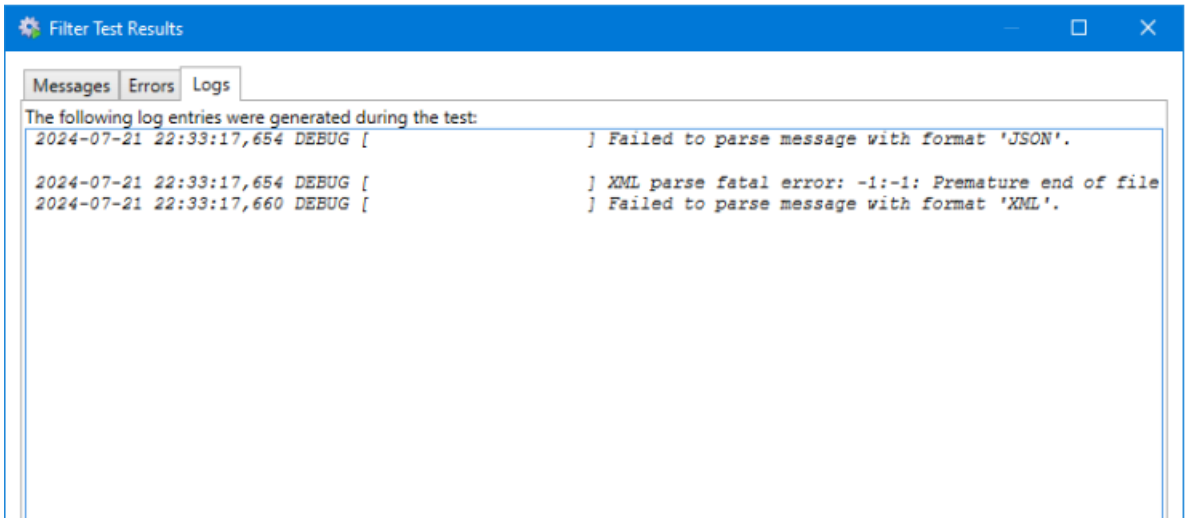
Errors

Selecting the test result's **Errors** tab identifies any errors that may have occurred during the processing of the test message by the filter of conditional connector. A review of the Exception details and the stack trace will often help identify the source of this error, which can then be corrected before re-running the test.



Logs

Selecting the **Logs** tab displays the entries generated during the test. The level of detail to include in the log is configured on the **Filter** or **Conditional Connector Testing** screen.

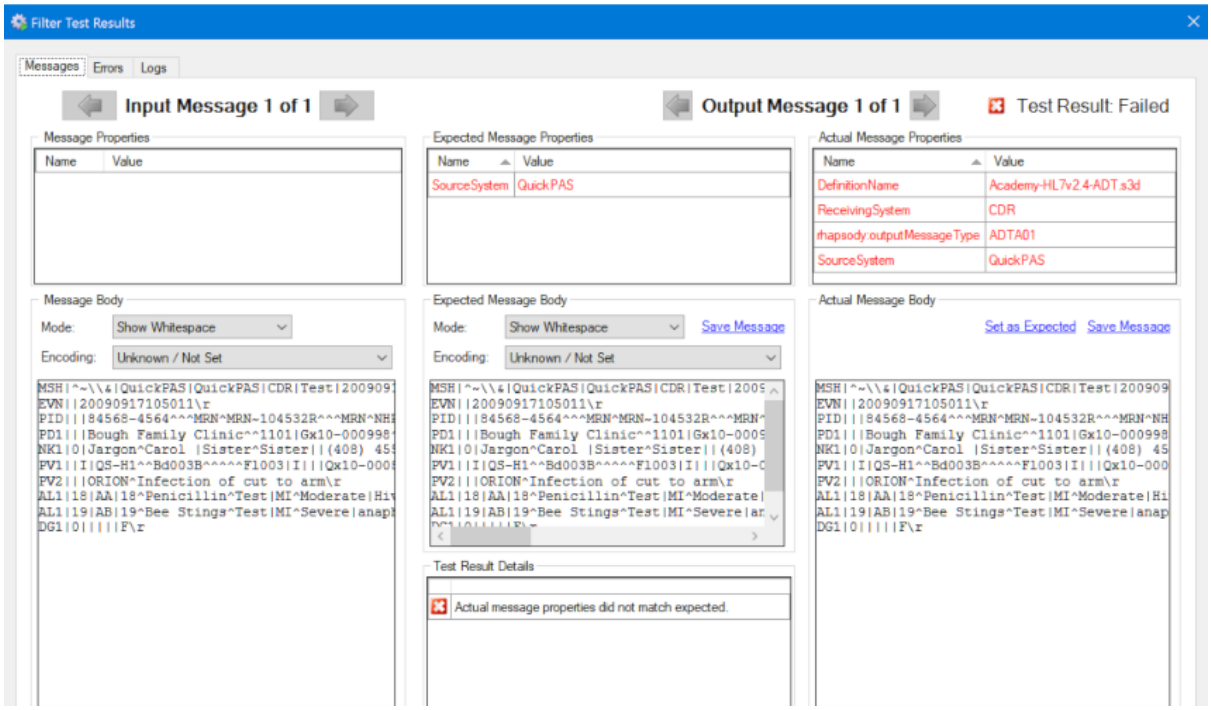


Only those log entries generated by the filter or conditional connector during testing are displayed on this screen. If other Rhapsody services are called during test execution, their log entries will not be displayed, but can be seen in the main Rhapsody log. The name of the sub-logger creating these entries will appear between the square brackets in the above image.

Testing Message Properties

When a filter is tested and the output message is validated, both the properties and content of the actual output message are compared with the properties and content of the expected output message. If both of these items are identical, the test result will be identified as a Pass.

When a test fails because the message properties do not match, the resulting screen will be similar to that shown below.



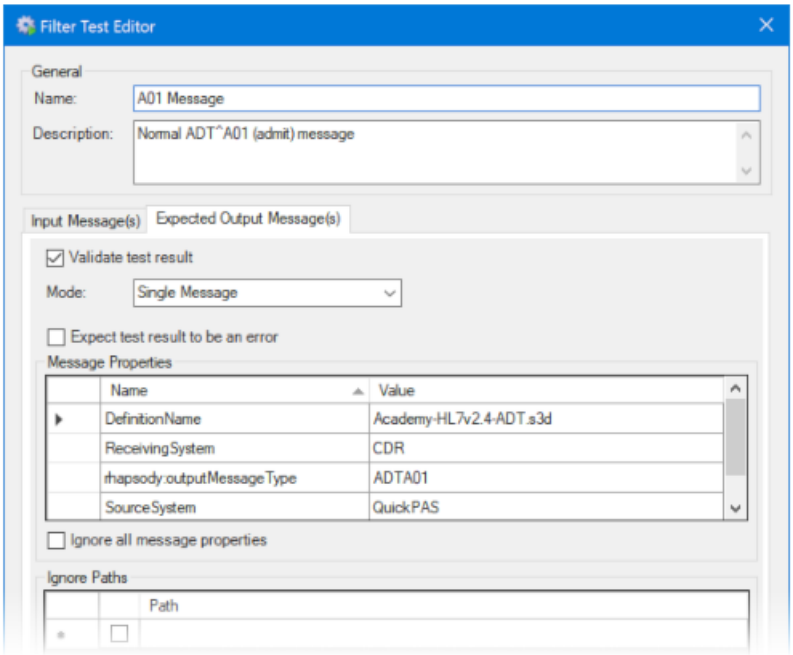
The **Test Result Details** panel identifies the reason for the failure of the test; in this case because 'Actual message properties did not match expected'.

The **Actual Message Properties** panel lists the messages associated with the output message following the test's completion. They are shown in red because at least one of them does not have a match in the **Expected Message Properties** panel.

Specifying the Message Properties

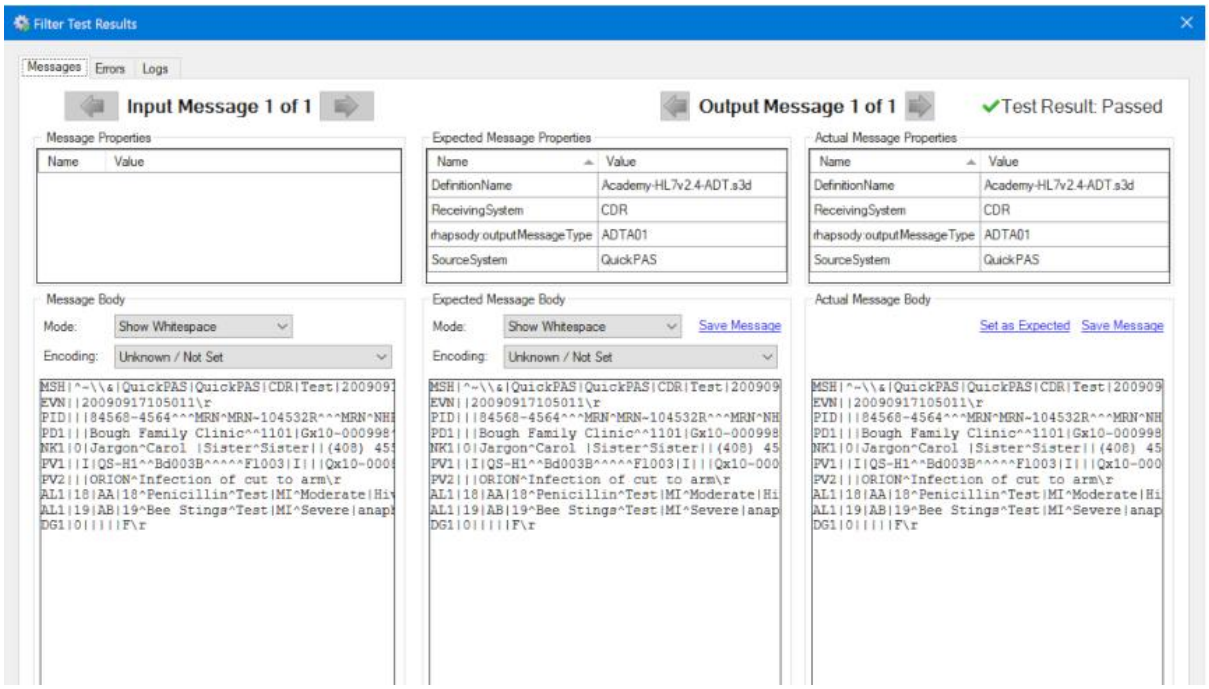
The message properties expected to be associated with an output message are manually entered into the **Expected Output Message(s)** tab of the Test Editor. The property names and values must exactly match the output from the filter, and all information is case-sensitive. Make sure the **Ignore all message properties** checkbox is not selected; in fact, you will not be able to add the properties if it is.

If a filter generates one or more message properties with an unpredictable value, the **Ignore all message properties** checkbox will be selected to prevent mismatched message properties from causing a test failure.



Message Properties Match - Test Passed

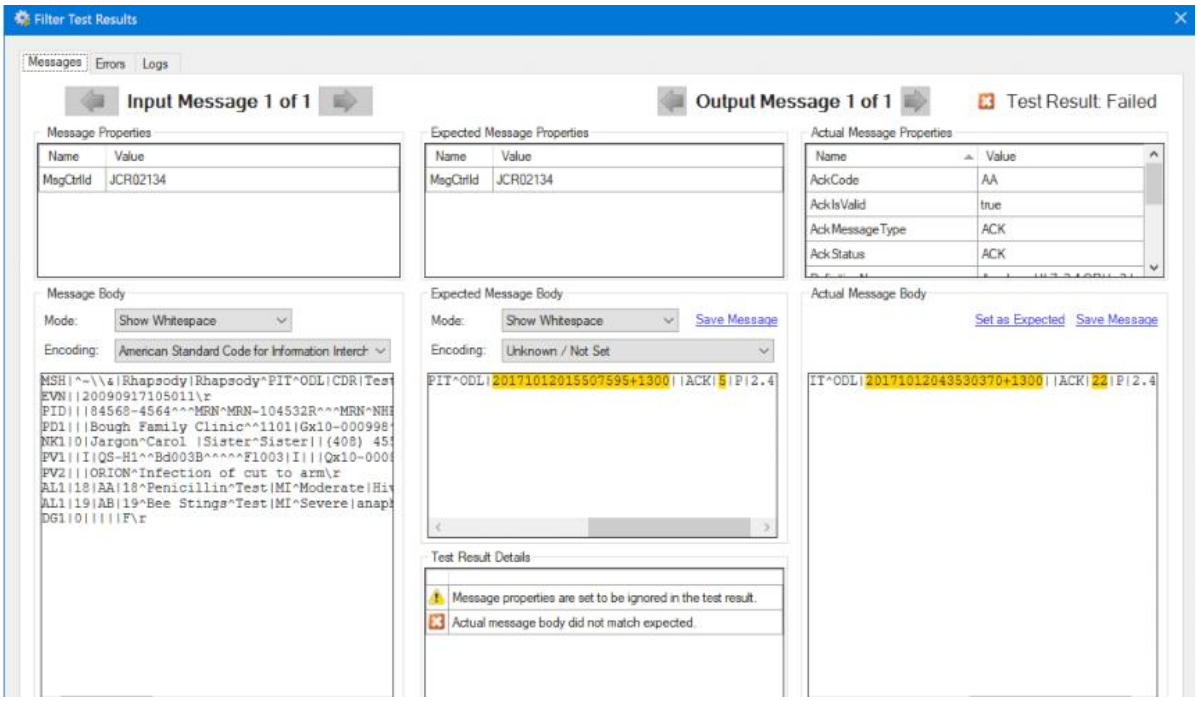
The result of a successful test, with the message properties now matching, is shown below:



Ignoring Message Fields

The following screenshot shows a test that has failed because the actual and expected message bodies do not match. In this case,

the MSH.DateTimeOfMessage and MSH.MessageControlID fields in the expected and actual messages do not match.

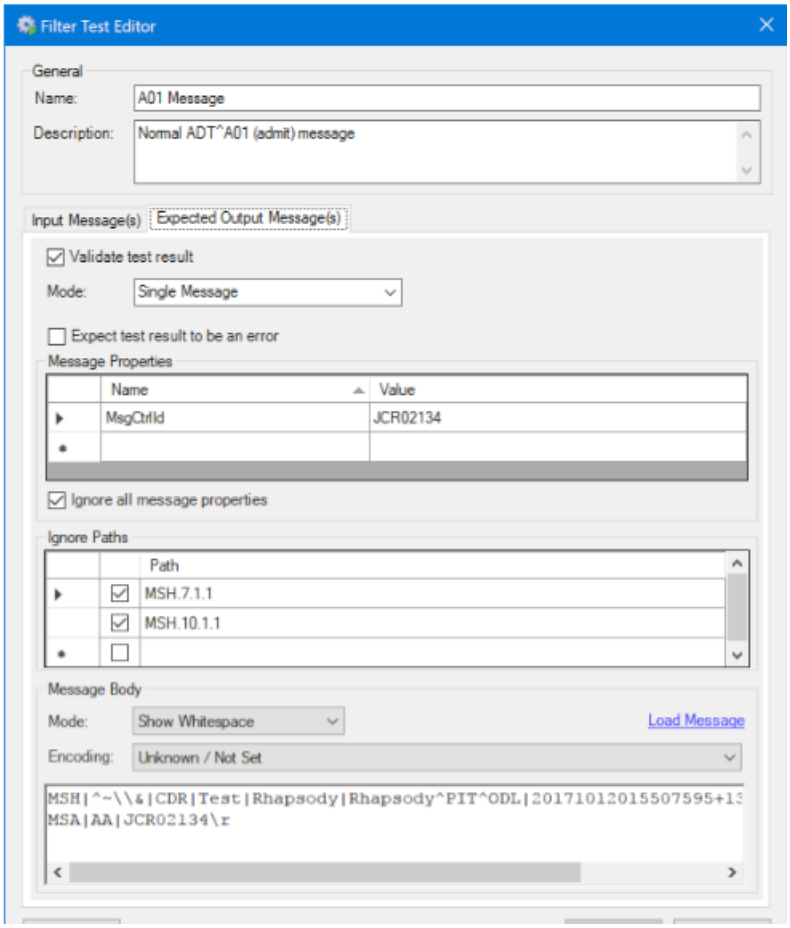


The reason for the test's failure is given in the **Test Result Details** panel. The differences between the expected and actual message body are highlighted. The test results panel also contains an alert message indicating that we are ignoring differences in message properties.

Specifying the Field Paths to Ignore

Where the message under test is HL7 version 2, we can specify one or more fields in the message to ignore during testing. The message fields to ignore are specified on the filter or conditional connector's Test Editor screen. In this example, the MSH.DateTimeOfMessage and MSH.MessageControlID fields have been specified to be ignored. Fields to ignore are specified with a HAPI style segment-dot-field number-dot-field number notation.

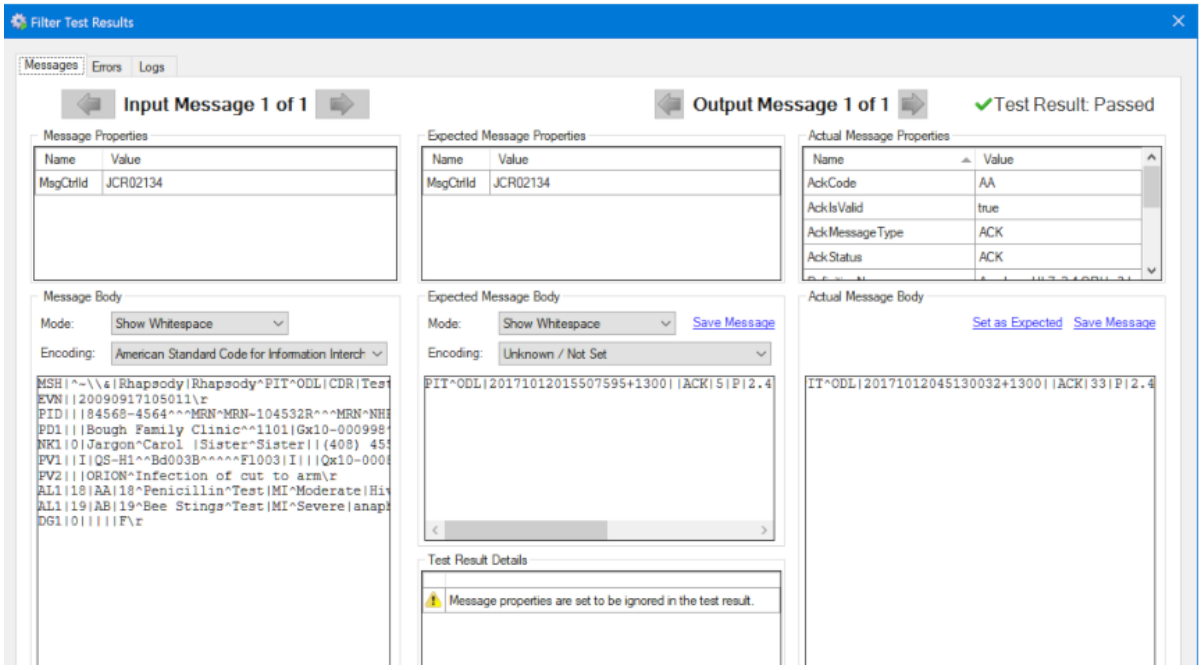
The checkbox to the left of each ignore path must be selected for the field in the message to be ignored during testing.



If a field is incorrectly entered, an Alert icon will be displayed in its row. The specified path will be not be ignored when the test is run until the path has been entered correctly.

Message Content Matches - Test Passed

The result of a successful test, with the content of the two message fields excluded from comparison, is shown below:



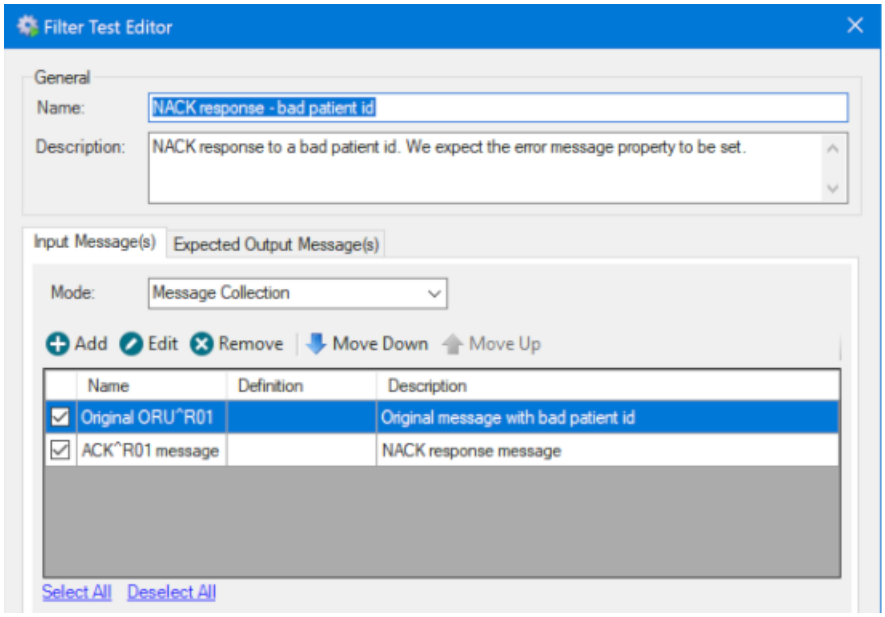
Testing Message Collections

Where a filter is expected to receive more than one message (via a message collector) or generate multiple messages, the Message Collection mode is used to configure test messages for the filter test. Conditional connectors only ever process one message at a time, so are always tested with individual test messages.

Input collections

Where a message collector that groups two or more messages have been configured, a message collector is configured for the input messages.

In the following example, a collection of messages has been specified as input to the filter. The test messages include a ORU^R01 and a corresponding ACK^R01 response. Notice the test name and description that clearly describe the test.



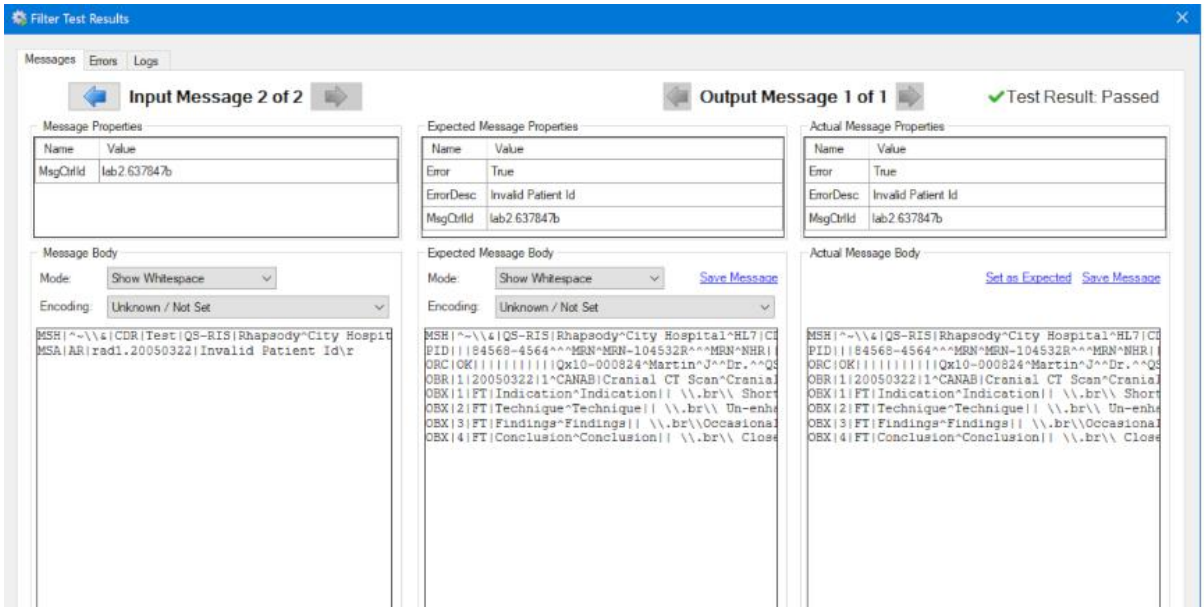
The Management icons are used to add, edit, and remove test messages from the list. When adding a test message, you can copy-paste an existing input test message from any filter or mapping project in Rhapsody by using the right-click mouse menu.

ⓘ The filter will process messages in the order that they appear in the Input Messages(s) set. When the output messages are being validated, the output message set needs to be in the same order as the as the messages generated by the filter.

Where the order of the input or output messages sets is important, use the **Move Up** and **Move Down** buttons to move a selected test relative to its neighbors.

Viewing the results of a Message Collection Test

The results show a filter that has used an input collation collector to group an HL7 message with its associated acknowledgement message and report on any errors found. The test was successful because both the expected and actual message properties and message body match.

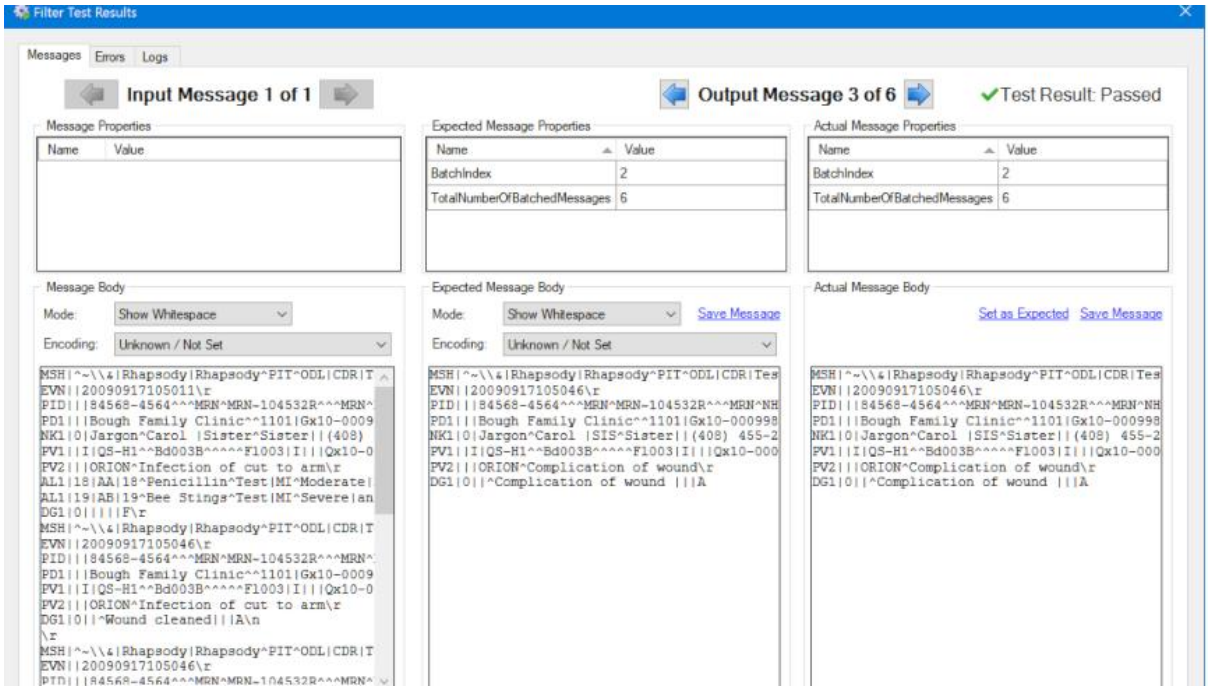


The **Input** navigation arrows are used to view each input in the test. In the example above, we are looking at the second message in the input set.

Output Collections

Where a filter has been configured to generate multiple output messages (for example, when using a de-batch filter to split up a message) an output collection is used to specify all the messages that the filter will generate.

In the following example, one input message has been split into six output messages. The screenshot shows output message 3 of 6 being displayed. As with the input example above, you can navigate between messages in the set using the left and right navigation arrows.

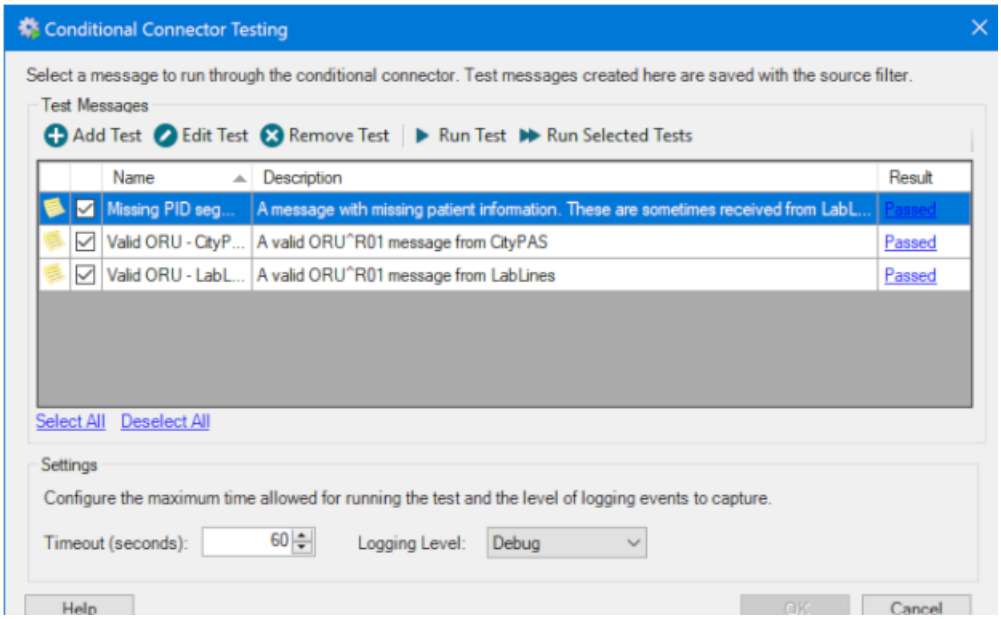


The BatchIndex property in the example above shows a value of 2 for output message number 3. This occurs because the Batch / De-batch filter uses a zero based index for this property.

Testing Conditional Connectors

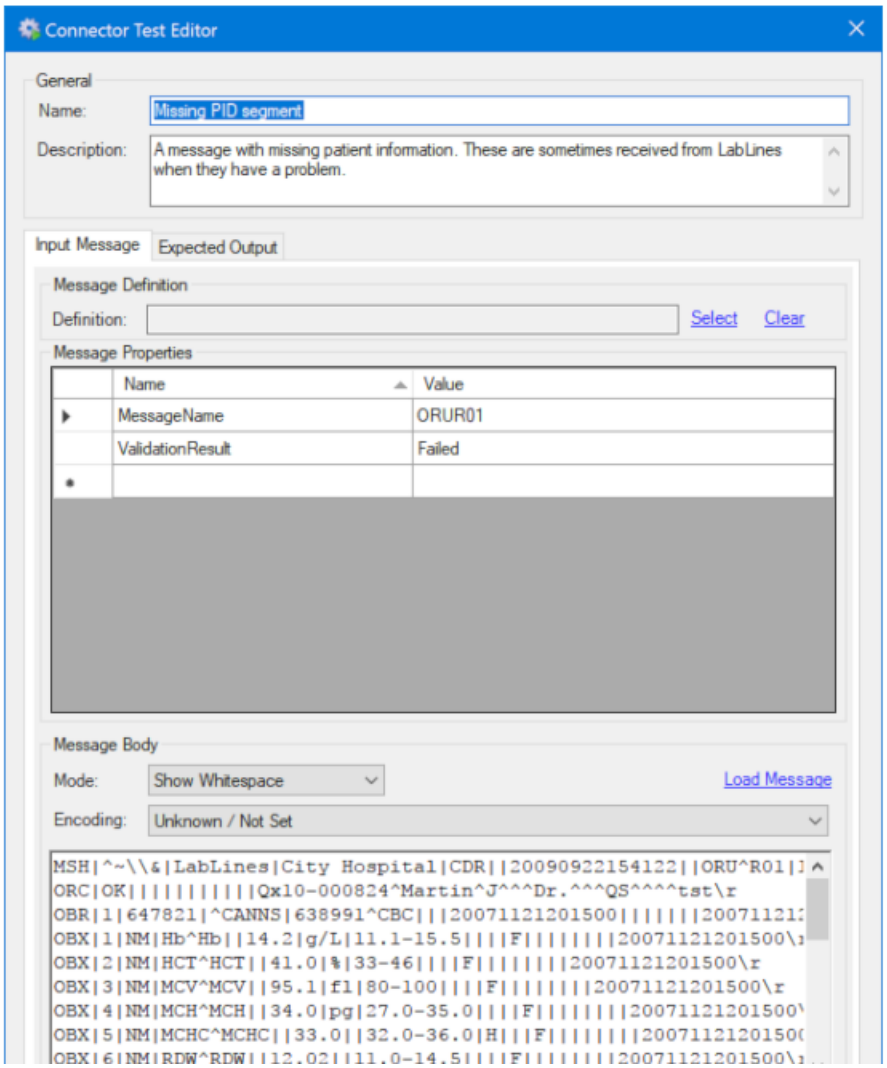
The process for testing conditional connectors is generally the same as filter testing. The process for creating test messages is identical. The expected output for a conditional connector will be (when validated) either 'Condition matched' or 'Condition not matched'. As a conditional connector can only process a single message at a time, there is no option to test multiple messages concurrently.

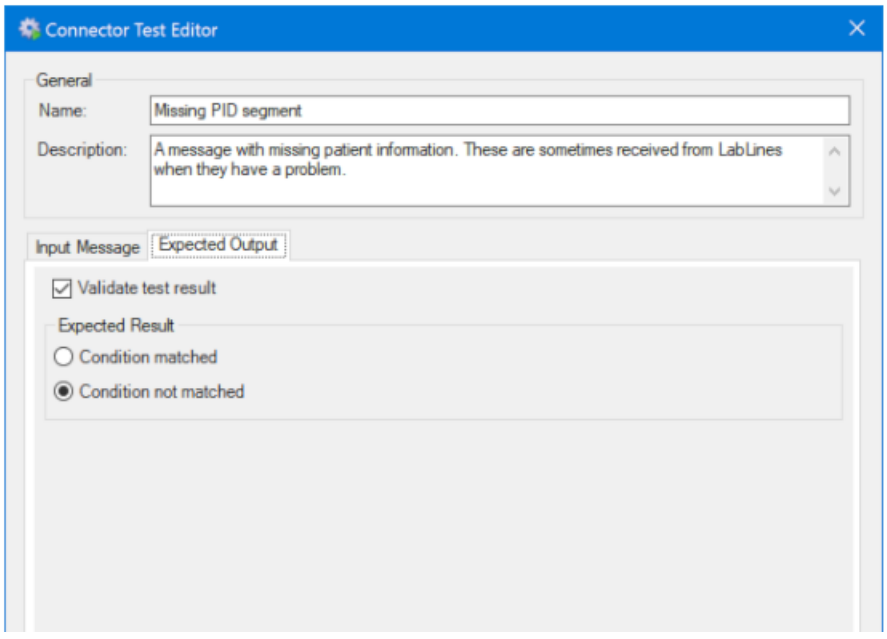
The following screen shows the results of testing the configuration of the conditional connector.



The Connector Test Editor

The Input Message tab presents a view identical to the single message test case for a filter. The Expected Output tab only provides options to indicate if a message will match the condition. Examples of these tabs are shown below:

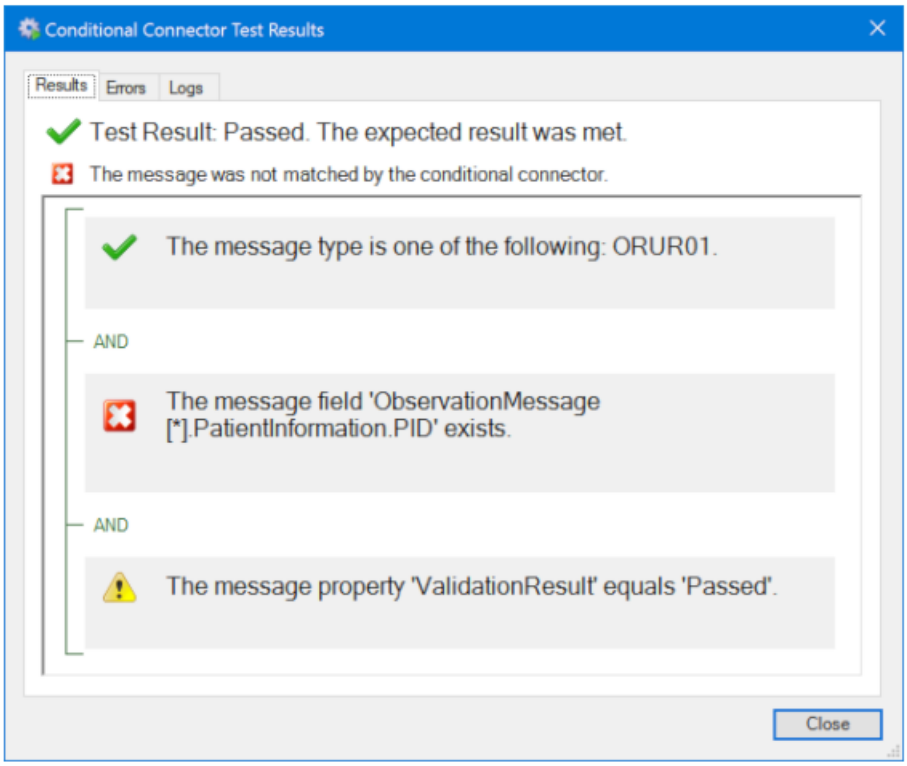




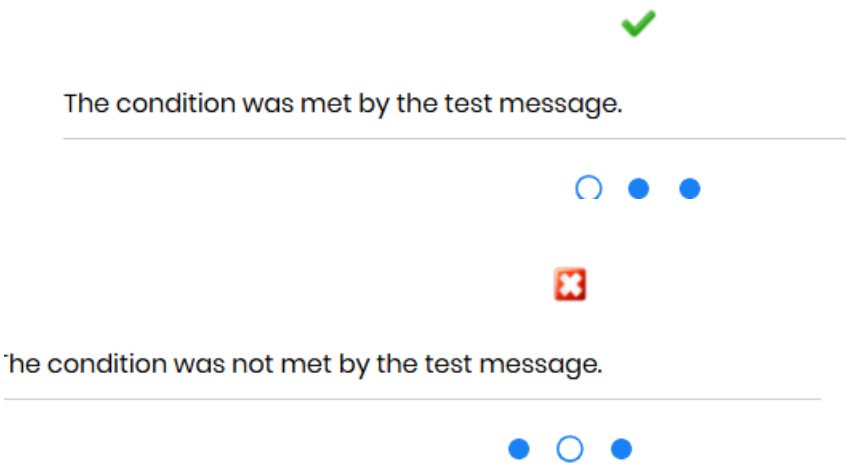
Test Results

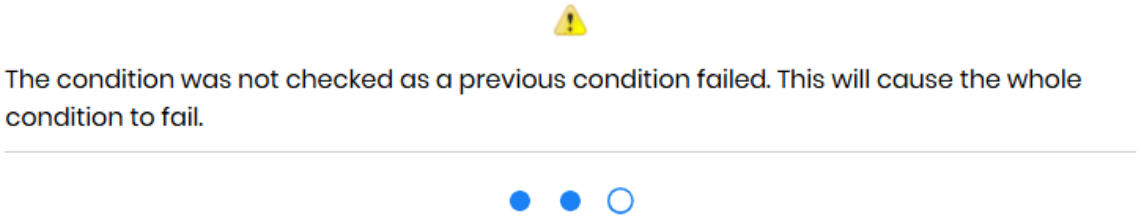
As with filter tests, selecting a completed test presents a summary of the action that would be performed on the test message by the conditional connector.

The results screen for a conditional connector provides details of the result of each condition that was checked on the test message, along with details on the overall outcome of the test.



The outcome of each condition is indicated with an icon as follows:

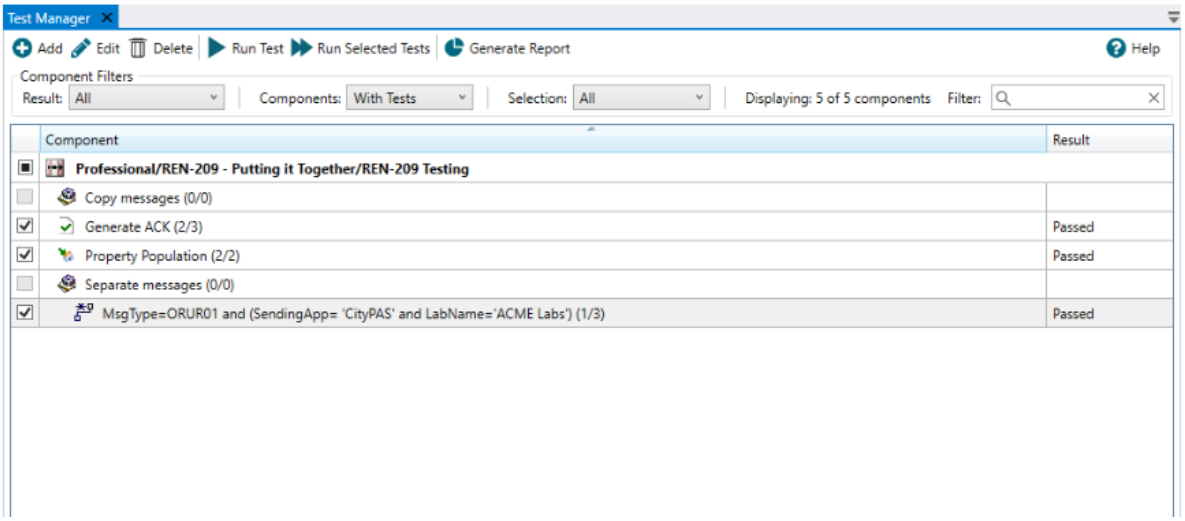




The **Errors** and **Logs** tabs function the same way as the equivalent tabs in the filter testing results screen.

The Test Manager

The **Test Manager** provides a centralized, hierarchical view of all filter and conditional connector tests, grouped by locker, in a Rhapsody solution. It is located in the Rhapsody IDE's **View** menu and the **Managers** drop down.



The view can be filtered to show only those components that match specified criteria, including:

- **Result** - the overall result for all checked tests associated with the filter or conditional connector – whether passed, failed, executed, skipped or none. The overall test result is determined by the highest priority result in the group of tests. For example, for a component with three tests where one failed and two passed, the cumulative result is failed.
- **Components** - include all components in the configuration, or only those with one or more associated tests.
- **Selection** - include all components with associated tests, or only those with one or more checked tests.

The count alongside the listed components identifies the number of associated tests (the second number) along with the number of those tests that are checked (the first

number). Note that the No-operation filter generally will not have an associated test, as it makes no change to messages as they pass through it. Also, some filters do not support testing, including the Duplicate Message Detection, Hold Queue, Database Look Up and Database Message Extraction filters.

Select the **Component** title to sort the filters and conditional connectors, grouped by route. Select the title a second time to reverse the sort.

Test Management

The tests associated with a selected filter or conditional connector can be managed by selecting the **Add**, **Edit**, or **Delete** icons at the top of the manager. Please review the previous pages in this module for details on completing the first two of these tasks.

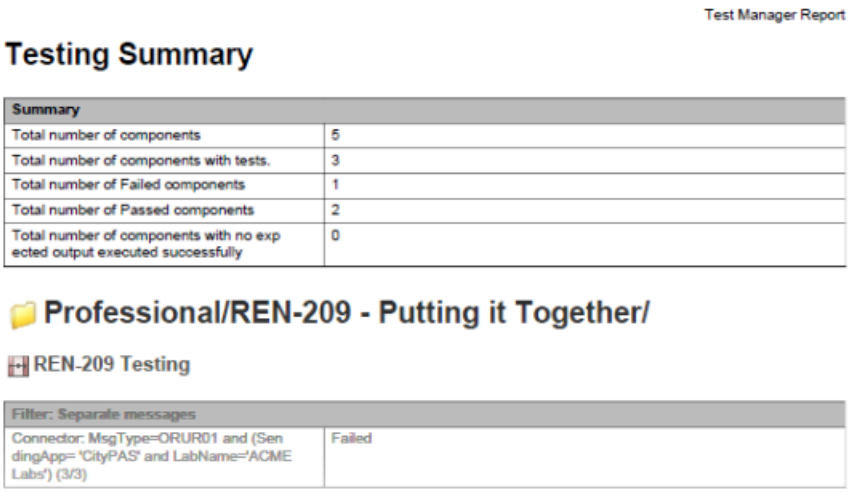
Run Tests, Run Selected Tests

Select a filter or communication point in the **Component** list, followed by **Run Test** to run that single test. Alternatively, select the component check-boxes followed by **Run Selected Tests** to run multiple tests simultaneously. Selecting or deselecting the checkbox alongside the route's title selects, or clears the selection of, all associated tests.

Selecting a component row opens the **Filter** or **Conditional Connector** Testing dialogs where its associate tests can be selected, edited and run.

Generate Report

Selecting Generate Report generates a PDF document containing a summary of the components with and without associated tests, along with the results of running those tests. The following screenshot shows a summary of the tests configured previously.



Test Manager Report

Testing Summary

Summary	
Total number of components	5
Total number of components with tests.	3
Total number of Failed components	1
Total number of Passed components	2
Total number of components with no exp ected output executed successfully	0

Professional/REN-209 - Putting it Together/

REN-209 Testing

Filter: Separate messages	
Connector: MsgType=ORUR01 and (Sen dingApp= 'CityPAS' and LabName='ACME Labs') (3/3)	Failed

The only option when generating the report is whether to include a Title Page. The report's footer includes the date and time on which the report was run, plus the Rhapsody version and the name of the Rhapsody server.